Information Sciences 690 (2025) 121586

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

PMformer: A novel informer-based model for accurate long-term time series prediction

Yuewei Xue, Shaopeng Guan *, Wanhai Jia

School of Information and Electronic Engineering, Shandong Technology and Business University, Yantai, 264005, China

A R T I C L E I N F O

Dataset link: https://github.com/BGCAI2023/PMformer

Keywords:
Long time series
Informer
Attention
Prediction

A B S T R A C T

When applied to long-term time series forecasting, Informer struggles to capture temporal dependencies effectively, leading to suboptimal forecasting accuracy. To address this issue, we propose PMformer, a novel model based on Informer for long-term time series prediction. First, we introduce a probabilistic patch sampling attention mechanism that utilizes a patch-based strategy to compute attention scores within randomly selected sequence patches. This localized approach enhances the model's capability to capture local temporal dependencies, allowing it to better understand and process critical local features in time series while reducing computational complexity. Additionally, we propose a multi-scale scaling sparse attention technique that balances attention distribution by combining coarse- and fine-grained attention scores, thereby improving the model's ability to capture global sequence information. Finally, we design a dilated causal pooling layer and a multilayer perceptual cross self-attention decoder to further enhance the model's prediction accuracy by capturing key information in long-term correlations and precisely focusing on sequences. We conducted experiments on both multivariate and univariate time series forecasting tasks. The results show that PMformer outperforms six baseline models, including PatchTST and FEDformer, in terms of MAE and MSE metrics. This demonstrates its superior ability to capture temporal dependencies, achieving more accurate predictions.

1. Introduction

A time series is a collection of data arranged in chronological order, recording the values or observations of variables at different points in time [26]. Forecasting, as one of the key tasks in time series analysis [16], plays a crucial role in medicine [18], stock market analysis [7], transportation [32], and other fields, providing important references for decision-makers. Therefore, accurate time series forecasting is of great importance [17]. Short-term time series forecasting usually involves predicting data over a relatively short time horizon in the future and is relatively easy to implement [11]. However, Long Series Time Series Forecasting (LSTF) requires more historical data to predict future conditions over a longer time horizon, and the prediction accuracy still needs improvement [34]. Existing methods for solving the time series forecasting problem include statistical methods [12] and machine learning models [3]. Statistical methods are poor at capturing nonlinear relationships and are difficult to handle complex time series data [29].

Among the common machine learning temporal prediction methods, recurrent neural networks (RNNs) and their variants long short-term memory (LSTM) and gated recurrent unit (GRU) are widely used to capture temporal dependencies in sequential data. However, RNNs and their variants face some key challenges when dealing with long sequences: first, these models are prone to suffer

* Corresponding author.
E-mail address: 201513035@sdtbu.edu.cn (S. Guan).

https://doi.org/10.1016/j.ins.2024.121586
Received 1 June 2024; Received in revised form 27 September 2024; Accepted 20 October 2024

Available online 23 October 2024

from the gradient vanishing and gradient explosion problems, which affect the effectiveness of modeling long-term dependencies; second, they are computationally inefficient when dealing with long sequences, which leads to a significant increase in the training and inference time; and lastly, since RNNs process data stepwise in sequences, this limits their parallel computing capability, which is particularly inconvenient in modern efficient computing environments. In addition, convolutional neural networks (CNNs) are also commonly used for time series prediction. Although CNNs perform well in areas such as image processing, their localized receptive fields are designed to make them challenging when dealing with time-series data. CNNs extract local features mainly through convolutional kernels and thus have limitations in capturing long-range temporal dependencies.

The field of time series forecasting has seen new growth opportunities with the widespread use of Transformer models [1]. Transformer is known for its excellent ability to capture long-range dependencies [24]. However, although Transformer can understand and generate text sequences well in the field of natural language processing, it suffers from high consumption of computational resources and slow inference in the field of time series prediction. This limits the wide application of Transformer models in real-time prediction and application scenarios. Optimization and improvement of Transformer can make it more practical and efficient in long time series prediction.

Informer is an improved model based on the Transformer [39], designed for long sequence prediction tasks. The model cleverly combines global and local self-attention mechanisms to effectively balance the capture of macro trends and micro details when analyzing the data, thus ensuring that the model can fully understand the complex dynamics of the sequences [35]. In addition, Informer employs a multi-layer encoder-decoder structure to delve deeper into the information in the sequence data through hierarchical processing and gradually improve the prediction accuracy. Notably, the Informer model also introduces a sparse attention mechanism, designed to optimize computational efficiency and model scalability. By streamlining the computational process and accelerating the processing speed, the mechanism enables the model to maintain efficient performance when dealing with complex sequences with long time spans.

Although Informer introduces global and local self-attention mechanisms to capture dependencies in a sequence, the balance between these two attention mechanisms is less than ideal. Global self-attention is mainly used to capture long-distance dependencies in a sequence, while local self-attention focuses more on local relationships in a sequence. However, in practice, global self-attention may pay too much attention to certain parts of the sequence, resulting in the neglect of some local relationships. Conversely, in sequences with complex structures or high stochasticity, local dependencies may not be obvious enough or may be irregular, leading to poor performance of the self-attention mechanism in learning and capturing local dependencies.

Specifically, this may occur in the following scenarios: first, if the data in the sequence has a high degree of randomness, such as more noise or irregular patterns, then the local dependencies may become ambiguous, or there may be no obvious local dependency structure. In this case, the self-attention mechanism may not be able to accurately capture important features in the sequence, leading to performance degradation. Second, in some sequences, although local dependencies exist, these dependencies may extend to farther locations and are not limited to a local scope. In this case, using only the local self-attention mechanism may not capture the dependencies between long-distance locations in the sequence, leading to a degradation in model performance.

To improve the computational efficiency and prediction accuracy of long sequence prediction models, a series of Transformer variant models have emerged. Autoformer [27] and Pyraformer [15] are two improved Transformer-based models that employ a direct multi-step prediction strategy. Despite the improved performance of these models on the Long Sequence Time Forecasting (LTSF) benchmark, the predictions are still inaccurate. FEDformer [40] is a neural network model based on frequency-domain decomposition that uses Fourier and wavelet transforms to apply global properties of the sequence to the frequency domain, but it ignores multi-scale local features in the time series. DLinear [33] combines convolutional neural networks and recurrent neural networks, using simple linear layers, and demonstrates superior performance to Transformer-based models in some cases. However, the emergence of PatchTST [20] challenged this conclusion. PatchTST is a patch-based neural network model that utilizes channel-independence to predict multivariate time series, which significantly improves the prediction performance. However, the model mainly relies on the global attention mechanism, which results in high computational complexity. MLP-Mixer [38] proposes the idea of using multi-scale patches, but there is still room for improvement in the integration and balance of multi-scale information. Pathformer [5] proposes using the attention mechanism between and within patches, resulting in high computational complexity when dealing with extremely long sequences.

Several recent studies have explored how to improve the Informer architecture, for example, MFTM-Informer [36], CEEMD-MsI [37], Spatiotemporal Informer [9], and EEMD-PSO-Informer [14]. These improved Informer models are optimized for nonlinear and nonsmooth data by signal preprocessing, pattern matching, decomposition, and reconstruction, aiming to improve time series prediction accuracy. However, these models suffer from the following shortcomings:

Ineffective balance between global and local attention: Many models either overemphasize global dependencies at the expense of local details or vice versa. This imbalance leads to suboptimal performance in capturing intricate temporal patterns.

Computational inefficiencies: The high computational complexity of attention mechanisms remains a challenge, particularly for long sequences.

Limited scalability and generalization: Models that perform well in specific domains may not generalize effectively to other areas or datasets.

To address the above problems and enable the model to better synthesize global and local information in sequences, we propose a correlation-enhanced Informer: PMformer for long-time sequence prediction. In it, the encoder utilizes local attention computation and causal convolution operation to capture the local time dependence and long-term correlation of the time series, while the decoder employs a multi-layer perceptual cross-self-attention decoding module to improve the accurate focusing and decoding performance of the key information. Our main contributions are reflected in the following aspects:

Probabilistic patch sampling attention mechanism: We devise a novel probabilistic patch sampling attention mechanism that significantly reduces computational complexity by randomly selecting sequence patches and computing attention scores only within these patches while enhancing the model's ability to capture local temporal dependencies.

Multi-scale scaling sparse attention mechanism: We design a multi-scale scaling sparse attention mechanism. This attention mechanism first computes coarse-region level attention scores, and then balances and adjusts the attention scores by combining these scores with additional fine-grained attention information to improve the model's ability to capture global dependencies in sequence prediction tasks.

Extensive domain validation: Our proposed model has been extensively tested on datasets from six different domains: finance, meteorology, energy, transportation, public health, and environmental monitoring, and the experimental results show that our method outperforms all the baseline methods in both MAE and MSE metrics, significantly improving the prediction accuracy.

The rest of the paper is organized as follows. Section 2 analyzes the related work, Section 3 describes the scheme proposed in this paper, Section 4 describes the experimental setup, and Section 5 summarizes the full work.

2. Related work

Improving the accuracy of long time series prediction has become a hot topic with the increasing interest in long time series prediction tasks [36]. Existing methods for solving the time series forecasting problem include statistical methods and machine learning models [37].

Traditional statistical methods, such as SARIMA [9] and ARMA [14], are still used, but they were originally designed based on deterministic estimation of time series parameter models. These models tend to perform poorly when dealing with long series time series forecasting tasks. A major reason for this is their assumption that future trend data are stable, which does not correspond to the real situation [26]. Moreover, when faced with large data samples, these traditional models usually fail to provide desirable predictions because they lack the ability to capture complex temporal dynamics.

In the face of these problems, machine learning models are beginning to capitalize on their unique strengths. Pattanayak et al. proposed a time series forecasting model based on SVM, which uses observations and average aggregated membership value to construct fuzzy logic relationships and improve data prediction accuracy [31]. Mozaffari et al. proposed a hybrid model based on SVR, which used a particle swarm optimization algorithm (PSO) to optimize the SVR parameters and accurately predicted the groundwater level in the aquifer [23]. Wang et al. proposed a time series prediction model based on Bayesian networks, which improves prediction accuracy by integrating fuzzy logic relationships and Bayesian modeling of dependencies for a comprehensive understanding of complex relationships in time series data [8]. Xu et al. proposed a retail price index forecasting method using a Gaussian process regression model, which provides decision-makers with a tool to assess retail real estate prices through Bayesian optimization and cross-validation [21]. However, as the demand for applications of machine learning methods grows, it is becoming increasingly important to perform more complex and fine-grained feature engineering on time series data. Especially when dealing with high-dimensional real-world data, the generalization ability of traditional machine learning methods is significantly limited. In addition, traditional machine learning methods usually focus only on extracting temporal features and ignore the intrinsic correlation between time steps, which poses a challenge for long-series time series prediction.

With the development of deep neural network technology, its powerful nonlinear modeling capability and ability to learn complex data relationships have shown great potential in processing time series data [22]. Compared to traditional statistical models and machine learning methods, neural networks are better able to capture temporal features and underlying patterns in time series data, leading to more accurate and effective predictions. Cascone et al. proposed a hybrid deep learning model that combines convolutional features with LSTM, which successfully predicts electricity consumption for the coming week using smart meter readings [19]. Xiang et al. proposed a hybrid model based on temporal convolutional networks and gated recurrent units, which solves the photovoltaic power prediction problem by performing spatial feature extraction from time-domain convolutional networks [25]. Aseeri proposed a lightweight recurrent neural network method that systematically preprocesses multivariate time-discrete power data to solve the problem of electric load forecasting [30]. Gao et al. proposed a hybrid model combining convolutional neural networks (CNN) and long short-term memory (LSTM), which investigated the response of normalized vegetation index (NVI) to climatic factors and improved the prediction accuracy of NVI [6]. Cai et al. proposed a deep learning model based on frequency domain analysis, which decomposed the time series into different time scales and solved the problem of multivariate time series prediction [4]. Liu et al. proposed a time series time-frequency domain decoupled representation model, which improved the accuracy and analytical ability of carbon emission prediction through decoupled representation comparative learning [28]. However, these models struggle to extract useful features when dealing with datasets that do not vary significantly in time scales, leading to a decrease in prediction accuracy [2].

In this context, Transformer-based methods have become a key technique for achieving better performance in time series forecasting tasks [10]. The attention mechanism allows these models to better capture correlations and crucial information between time steps, thus improving prediction accuracy and generalization. Informer is one such Transformer-based model specifically designed for long sequence time series prediction. By introducing sparse attention and a self-attentive distillation mechanism, Informer enhances prediction efficiency and performance, effectively handling long sequence data. Several studies have explored improvements upon the Informer architecture. Zhao et al. [36] proposed a multi-step forecasting model called MFTM-Informer, which reconstructs data using sample entropy to combine trends and fluctuations for prediction. This approach addresses the challenges of multi-step forecasting in areas such as disaster warning and financial analysis. Zheng et al. [37] proposed a CEEMD-MsI model based on complete ensemble empirical mode decomposition. This model processes data from different geographical locations and times through multiple data

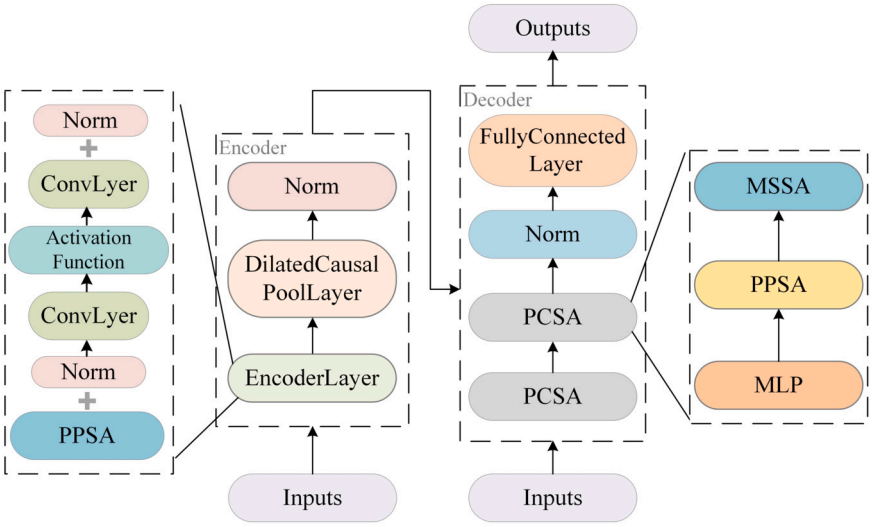

Fig. 1. The architecture of PMformer.

streams, effectively predicting PM2.5 concentrations. Feng et al. [9] introduced a Spatiotemporal Informer method, which leverages spatiotemporal embedding and attention to enhance the accuracy of air quality index predictions. Furthermore, Zheng et al. [14] proposed a hybrid EEMD-PSO-Informer model, combining ensemble empirical mode decomposition with a particle swarm optimization algorithm for parameter optimization. This approach effectively addresses the nonlinearity and non-stationarity of building energy consumption data. While Informer has demonstrated excellent performance in capturing remote dependencies using the self-attention mechanism, the balance between global and local attention mechanisms remains suboptimal.

To address the above problems, we propose a correlation-enhanced Informer for long time series prediction tasks. We design a probabilistic patch sampling attention mechanism and a multi-scale scaling sparse attention mechanism, aiming to balance the capture of global and local dependencies and significantly improve computational efficiency and prediction accuracy.

3. Method

To efficiently extract features from input sequences and capture temporal correlations for accurate long sequence prediction, we designed a long time sequence prediction model based on an encoder-decoder architecture: PMformer. Its overall architecture is shown in Fig. 1:

PMformer reads the dataset file stored in the data frame and selects the data containing the date column, feature column, and target column. The data is then divided into training, validation, and test sets. The feature column data are normalized so that the model can be better trained. Next, timestamped features are generated by a time coding function to provide time information to the model. The data go to the encoder section.

In the encoder, data first pass through an embedding layer and are transformed into a fixed-length vector representation. Subsequently, these vectors are passed to multiple encoder layers. In each encoder layer, the probabilistic patch sampling attention (PPSA) mechanism is first applied, which reduces the computational complexity and enhances the ability to capture local temporal dependencies by calculating the attention scores within different sequential segments. Meanwhile, the causal convolutional extension module captures long-term correlations at different time scales and outputs high-dimensional feature representations by further processing the features and normalizing them through extended convolutional operations.

In the decoder portion, the decoder receives output features from the encoder as well as input data from the decoder, which are transformed into fixed-length vectors via a data embedding layer. Then, multiple decoder layers in the decoder process these vectors. In each decoder layer, the input data are first nonlinearly transformed, and features are extracted by a multilayer perceptron (MLP) to enhance the abstract representation of the model. Next, a probabilistic patch sampling attention mechanism performs attention computation on different time steps within the input data in order to capture key information in the sequence. The processed data are further optimized through residual joining and layer normalization. In addition, the multi-scale scaling sparse attention mechanism computes the attentional weights between the decoder input data and the encoder outputs, integrating the contextual information provided by the encoder, which improves the understanding of the input sequence. This process is repeated for each decoder layer, progressively optimizing the feature representation of the input data. Finally, the processed data are mapped to the final output space through a linear projection layer, generating the predictions of the model. In this way, the decoder is able to accurately capture complex dependencies in the time series and provide high-quality predictions.

Taken together, the PMformer model utilizes the interaction between the encoder and the decoder to efficiently extract features from the input sequence, capture temporal dependencies, and achieve accurate time series predictions.

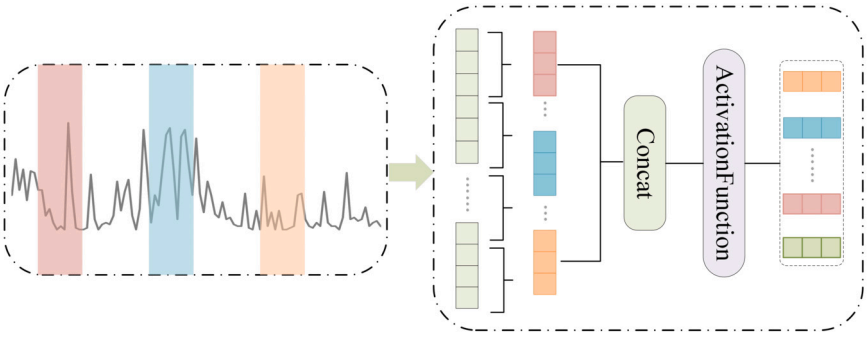

Fig. 2. The architecture of PPSA.

3.1. Probabilistic patch sampling attention

When facing long sequence time series prediction problems, traditional global attention mechanisms are often limited by high computational resource consumption and the inability to effectively capture local time dependencies in the sequences. To address these challenges, we propose probabilistic patch sampling attention (PPSA), which optimizes the efficiency of attention computation and enhances the model's ability to capture local dependencies by introducing random patch sampling and dynamic pruning techniques, thus reducing computational complexity. Its structure is shown in Fig. 2:

The core idea of PPSA is that instead of performing global attention computation on the entire sequence in each forward propagation, multiple patches of the sequence are randomly selected for local attention computation. We first initialize an all-zero score matrix, as shown in equation (1):

$$\text{scores} \leftarrow 0 \in \mathbb{R}^{B\times H\times L\times L} \tag{1}$$

where B is the batch size, H is the number of heads, and L is the sequence length. Initially, the score matrix is set to all zeros. The system then randomly selects a plurality of patches defined by randomly generated starting points, where each starting point defines a patch. Within these patches, the computation of queries, keys, and values is performed for each element in the input sequence, and local attention scores are generated. This process is shown in equation (2):

$$\text{scores}[b,h,i,j] = (\mathbf{q}_{b,h,i} \cdot \mathbf{k}_{b,h,j}) \quad \text{for } i,j \in [\text{start},\text{end}] \tag{2}$$

where $\mathbf{q}_{b,h,i}$ and $\mathbf{k}_{b,h,j}$ represent the query and key vectors for locations i and j under batch b and head h, respectively. This localized computation significantly reduces the number of data points involved, decreases the demand for computational resources, and enhances the model's ability to capture key local time dependencies.

Next, a dynamic pruning technique is introduced to dynamically prune the score matrix. The absolute value and threshold are first calculated, and then thresholding is applied, as shown in equations (3) and (4):

$$\text{threshold} = \text{quantile}(|\text{scores}|, \text{prun\_rate}, \text{dim} = -1) \tag{3}$$

$$\text{scores} = \begin{cases} \text{scores} & \text{if } |\text{scores}| \geq \text{threshold} \\ -\infty & \text{otherwise} \end{cases} \tag{4}$$

Attention scores are dynamically adjusted according to the set pruning rate by filtering out lower scores and setting them to negative infinity, which effectively reduces computational complexity and lightens the model burden. This step determines which scores should be pruned by presetting thresholds to optimize the effectiveness of each computation and increase the sparsity of the model, further reducing computational and storage costs. Finally, the attention scores are normalized by applying the Softmax function, and the output values for each position are calculated by weighted summation, as shown in equations (5) and (6):

$$A = \text{Softmax}(\text{scores}, \text{dim} = -1) \tag{5}$$

$$V = \text{einsum}('bhls, bshd \rightarrow blhd', A, v) \tag{6}$$

where A is the attention weight matrix with the shape (B, H, L, S) and v is the value matrix with the shape (B, S, H, D). By the Einstein summation convention, A and v are element-wise multiplied and summed along the dimension S to obtain an output matrix V of the shape (B, L, H, D). This operation applies the attention weights A to the value matrix v and performs a weighted summation of the elements over the length of the sequence of keys S to obtain the output value V at each position.

By combining random patch sampling and dynamic pruning techniques, PPSA allows the model to focus more efficiently on local information critical to the prediction task when dealing with large-scale sequence data, avoiding unnecessary global computations. This strategy not only significantly improves the model's processing capability in long sequence tasks but also enhances computational performance and storage efficiency, making the model more scalable and practical in real-world applications.

The process of constructing the probabilistic patch sampling attention mechanism for PMformer is shown in Algorithm 1:

Algorithm 1 Probabilistic patch sampling attention.

1: **Input:**
2: Queries Q
3: Keys K
4: Values V
5: Attention Mask (optional)
6: `mask_flag`
7: `patch_size`
8: `num_patches`
9: `attention_dropout`
10: `output_attention`
11: `prune_rate`
12: **Output:**
13: Output V
14: Attention weights A (optional)
15: Initialize scores S as zeros
16: **for** each patch (`num_patches` times) **do**
17: Randomly select start and end indices
18: **for** each position (i, j) in patch **do**
19: Compute $S[i, j]$ as dot product of $Q[i]$ and $K[j]$
20: **end for**
21: **end for**
22: Compute absolute values of S
23: Apply pruning by setting scores below threshold to $-\infty$
24: **if** `mask_flag` is True and mask is provided **then**
25: Apply mask to S
26: **end if**
27: Apply softmax and dropout to S to obtain A
28: Compute V as weighted sum of V using A
29: **if** `output_attention` is True **then**
30: Return (V, A)
31: **else**
32: Return V
33: **end if**

Algorithm 1 outlines the probabilistic patch sampling attention mechanism. The algorithm first defines the input parameters, including queries (Q), keys (K), values (V), an optional attention mask, and configuration parameters (Lines 2-11). A scores matrix, S, is then initialized to zero (Line 15), and the algorithm iterates over a predefined number of patches (Lines 16-21). For each patch, start and end indices are randomly selected, and attention scores are calculated for all positions within the patch. The algorithm then computes the absolute values of S and implements pruning by setting scores below a predefined threshold to $-\infty$ (Lines 22-23). Conditional masking of S is performed if `mask_flag` is set to True and a mask is provided (Lines 24-26). Subsequently, softmax and dropout are applied to S to obtain the attention weights, A (Line 27). Finally, the output V is computed as a weighted sum of values using A (Lines 28-33). The algorithm returns either both V and A, or just V, depending on the value of the `output_attention` flag.

3.2. Dilated causal pooling layer

The dilated causal pooling layer is designed to efficiently capture long-term correlations between time series at different levels of granularity. In time series data, long-term correlations may exist between different time steps, and the information at the current time step may be influenced by several time steps in the past. To better understand and utilize such long-term correlations, the introduction of the dilated causal pooling layer enables the model to more accurately capture key features and patterns in the sequence data, thereby improving the model's performance in tasks such as time series forecasting. The design idea of this module is based on the causal convolution operation, as shown in equation (7):

$$y[n] = \sum_{k=0}^{M-1} x[n + \text{padding} - k] \cdot w[k] + b \tag{7}$$

where $y[n]$ is the n-th element of the output sequence, x is the input sequence, w is the convolution kernel, and b is the bias. By setting a special padding and convolution kernel, the convolution operation is restricted to access only past inputs and not future inputs. This design ensures that the model considers past information sequentially in chronological order when performing convolution calculations without being affected by future information, thus better modeling the causality of time series data. Additionally, the module improves the stability and nonlinear representation of the model by using batch normalization and ELU activation functions. Finally, the main features are extracted by a max pooling operation, as shown in equation (8):

$$x_{\text{max\_pool}}[i] = \max(x[i : i + \text{kernel\_size}]) \tag{8}$$

where $x_{\text{max\_pool}}[i]$ is the output of max pooling, x is the input sequence, and kernel_size denotes the size of the pooling window. The max pooling operation downsamples the feature maps by selecting the maximum value within each pooling window to reduce

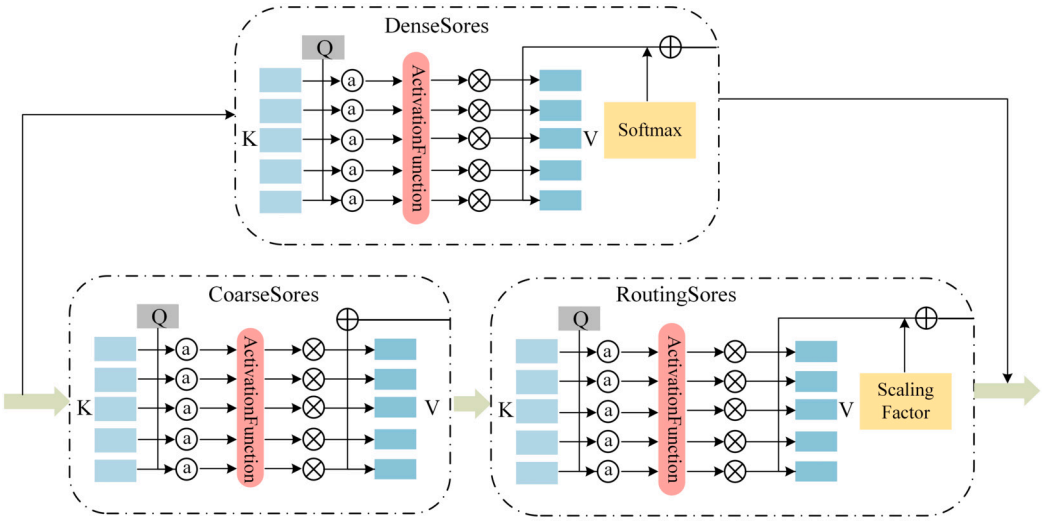

Fig. 3. The architecture of MSSA.

computation and the number of parameters, and to enhance the translation invariance of the model. The design of the dilated causal pooling layer enhances the PMformer model's ability to model sequence data, especially in capturing long-term correlations at different granularity levels more effectively when dealing with time series data. By handling temporal causality in a rational manner, the module helps to improve the model's understanding and prediction accuracy of time series data, thus enhancing the performance of the PMformer model in time series prediction tasks.

3.3. Multi-scale scaling sparse attention

The multi-scale scaling sparse attention (MSSA) aims to enhance the model's ability to capture the global information of the sequence. Its structure is shown in Fig. 3:

The mechanism is realized through two key steps: coarse region-level attention computation and fine-grained token-to-token attention. First, with coarse region-level attention computation, the model can quickly capture global sequence information. Attention scores at the coarse region level are calculated as shown in equation (9):

$$\text{coarse\_scores}_{bhls} = \text{queries}_{blhe} \times \text{keys}_{bshe} \tag{9}$$

where queries_{blhe} is the query, keys_{bshe} is the key, b denotes the batch size, l denotes the length of the query sequence, h denotes the number of queries and keys, e denotes the feature dimensions of the query and key, s denotes the length of the sequence of values, and d denotes the feature dimensions of the values. This step uses dynamic sparse attention, which performs a product operation on the query and key and applies a softmax function to compute the initial routing score, as shown in equation (10):

$$\text{routing\_scores} = \text{softmax}(\text{coarse\_scores})^{\text{factor}} \tag{10}$$

where the factor is used to amplify the initial routing scores to improve sparsity. These scores are amplified to improve sparsity and are combined with additional dense scores to balance the distribution of attention scores. The balanced attention process is shown in equations (11) and (12):

$$\text{combined\_scores} = \frac{\text{routing\_scores} + \text{dense\_scores}}{2.0} \tag{11}$$

$$\text{dense\_scores} = \text{softmax}(\text{coarse\_scores}) \tag{12}$$

Next, these attention scores are utilized to calculate the final attention weights, which are applied to the values to obtain the final output. The calculation process is shown in equations (13) and (14):

$$V_{blhd} = \sum_{l=1}^{L} \text{sparse\_scores}_{bhls} \times \text{values}_{bshd} \tag{13}$$

$$\text{sparse\_scores}_{bhls} = \frac{\text{combined\_scores}_{bhls}}{\sum_{l=1}^{L} \text{combined\_scores}_{bhls}} \tag{14}$$

where l denotes each position in the sequence and the value of l ranges from 1 to L, representing a weighted summation of the attentional weights and values for each position in the sequence.

The multi-scale scaling sparse attention mechanism is implemented in a way that utilizes a combination of dynamic sparse and dense attention, allowing the model to quickly capture sequence information globally and improve the sparsity of attention by scaling up the score, thus reducing computational complexity. At the same time, dense attention provides additional information to balance the distribution of attention scores, making the model more flexible in learning the relationships between sequences. This extra information includes the correlation between different parts of the sequence, importance, etc., which helps the model better understand the structure of the sequence. In long sequence prediction tasks, this mechanism can help the model capture dependencies over longer distances. With coarse region-level attention computation, the model can quickly capture sequence information on a global scale, while fine-grained token-to-token attention allows the model to explore local relationships between sequences more deeply. This combination of global and local attention helps the model better understand the structure of sequence data and improves the model's performance in long sequence prediction tasks.

The process of constructing a multi-scale scaling sparse attention mechanism for PMformer is shown in Algorithm 2:

Algorithm 2 Multi-scale scaling sparse attention.

1: **Input:**
2: Queries Q
3: Keys K
4: Values V
5: Attention Mask (optional)
6: `mask_flag`
7: `factor`
8: `scale` (optional)
9: `attention_dropout`
10: **Output:**
11: Output V
12: Obtain dimensions of queries Q: (B, L, H, E) and keys K: (B, S, H, E)
13: Obtain dimensions of values V: (B, S, H, D)
14: **if** `scale` is None **then**
15: Set `scale` $\leftarrow \frac{1}{\sqrt{E}}$
16: **end if**
17: Compute coarse scores: $coarse\_scores \leftarrow \texttt{einsum}(Q, K)$ (shape: (B, H, L, S))
18: **if** `mask_flag` is True and an attention mask is provided **then**
19: Apply mask to $coarse\_scores$, setting masked elements to $-\infty$
20: **end if**
21: Compute routing scores:
22: $routing\_scores \leftarrow \text{softmax}(coarse\_scores)$
23: $routing\_scores \leftarrow routing\_scores^{\texttt{factor}}$
24: Compute dense scores:
25: $dense\_scores \leftarrow \texttt{einsum}(Q, K)$ (shape: (B, H, L, S))
26: Combine routing and dense scores:
27: $combined\_scores \leftarrow (routing\_scores + dense\_scores)/2$
28: Normalize combined scores:
29: $sparse\_scores \leftarrow combined\_scores/\text{sum}(combined\_scores)$
30: Compute weighted values:
31: $V \leftarrow \texttt{einsum}(sparse\_scores, V)$ (shape: (B, L, H, D))
32: **return** V

Algorithm 2 details the construction of a multi-scale scaling sparse attention mechanism. The algorithm begins by defining the input parameters and output, including queries (Q), keys (K), values (V), an optional attention mask, and additional configuration parameters (Lines 2-9). The dimensions of the input matrices are then retrieved (Lines 12-13), and if no scaling factor (scale) is provided, it is set to $\frac{1}{\sqrt{E}}$ (Lines 14-15). Coarse attention scores are then computed (Line 17) by multiplying Q and the transpose of K (K^T). If a mask is specified by mask_flag and provided, the corresponding elements within the coarse_scores are masked by setting them to $-\infty$ (Lines 18-20). Finally, routing scores are calculated (Lines 21-23), first by applying softmax to the coarse_scores, and then by raising the result to the power of the predefined factor. Next, dense scores are computed (Lines 24-25) by applying softmax to the dot product of queries (Q) and keys (K^T), resulting in dense scores with the shape (B, H, L, S). The algorithm then combines routing and dense scores (Lines 26-27) by averaging them. Specifically, the routing scores and dense scores are summed and divided by 2 to produce the combined scores. The combined scores are normalized (Lines 28-29) by dividing by their sum along the last dimension, resulting in sparse scores. Finally, the weighted values are computed (Lines 30-32) by applying the sparse scores to the values V, producing the output V with the shape (B, L, H, D). The algorithm then returns this output.

3.4. Multilayer perception cross self-attention decoding module

To enhance the performance of the Informer model in sequence prediction tasks, we designed a multilayer perception cross self-attention decoding module (PCSA) that includes an MLP, probabilistic patch sampling attention, and a multi-scale scaling sparse attention mechanism. This module aims to improve the model's feature extraction and modeling capabilities for input sequences, thereby enhancing the model's performance in long sequence prediction tasks.

When the MLP is introduced, the model can utilize the multilayer perceptron to perform nonlinear transformations and feature extraction. This allows the model to more fully understand the complex features in the input sequence and capture the abstract features in the sequence more efficiently. The MLP acts similarly to a feature extractor, transforming the original input sequence into a higher-level, more representative feature representation, helping the model better understand the structure and content of sequence data. In PCSA, probabilistic patch sampling attention and multi-scale scaling sparse attention further enhance the model's ability to focus on and process important information in the sequence. Probabilistic patch sampling attention improves the computational efficiency of the model and reduces computational and storage overhead by limiting the scope of the attention computation to focus on capturing local dependencies in the sequence. Intelligent pruning techniques make the model's attention more sparse, meaning it only focuses on a small number of important positions in the sequence, which further improves the model's efficiency and generalization ability. On the other hand, multi-scale scaling sparse attention balances the attention scores by introducing additional information that allows the model to better capture the global information in the sequence. By considering information at different scales, the model can understand the structure and content of the sequence more comprehensively, improving the model's ability to capture the global information of the sequence. This design makes the model more efficient in handling large-scale sequence data and improves the model's performance in long sequence prediction tasks.

Compared with the original decoder, the advantage of PCSA is that it improves the model's ability to model and process sequence data. First, through the introduction of the MLP, the model can learn and utilize the abstract feature representations in the input sequences more fully, improving the feature extraction capability of the model. Second, probabilistic patch sampling attention and multi-scale scaling sparse attention further enhance the model's ability to capture the dependencies of the sequence data, enabling the model to show better performance in long sequence prediction tasks.

4. Evaluation

In this section, we detail the experimental setup, results, and analyses. We begin by introducing the datasets, baseline models, and evaluation metrics employed in our experiments. Next, we validate the effectiveness of the proposed model across six datasets, providing a detailed analysis of the results. To further understand the contribution of each component to the model's performance, we conduct ablation studies. Finally, we explore the effects of different parameter settings on model performance through parameter sensitivity experiments. All experiments were conducted using Python 3.8 on a system equipped with an NVIDIA GeForce RTX 4060 Laptop GPU.

4.1. Datasets

To comprehensively evaluate the performance and generalizability of the PMformer model, we selected six public datasets covering several key domains, such as finance, meteorology, energy, transportation, public health, and environmental monitoring. The diversity and complexity of these datasets enable us to comprehensively validate the generalization ability and robustness of the model. All data are available from https://www.kaggle.com/ and https://archive.ics.uci.edu/.

NFLX: This dataset provides 2,517 pieces of data related to Netflix's stock price from 2023 through 2024, featuring various columns: date, opening price, high price, low price, closing price, and more. This dataset is used to forecast Netflix's stock price trends.

Seattle-weather: This dataset includes dates, maximum and minimum temperatures, and wind speeds, which are used to predict weather conditions.

Solarenergy: This dataset consists of solar data that can be used to predict future energy consumption. It contains data from April through August 2020 and consists of 7 columns.

Traffic_volume: This dataset contains hourly westbound Interstate 94 traffic volumes at DoTATR Station 301 in Minnesota, with a total of 48,205 pieces of data.

Pox: This dataset is derived from weekly varicella cases in Hungary and consists of a county-level adjacency matrix and time series of reported cases at the county level between 2005 and 2015, with 21 columns.

Departure: This dataset includes lighthouse monitoring data from British Columbia waters using daily observations of seawater temperature and salinity collected at or near high tide during the day for the period from 1914 to 2023.

4.2. Baseline methods

To provide a comprehensive evaluation of our models, we have selected a series of benchmark models that represent the current mainstream techniques and classical models for time series forecasting. These benchmark models are described below:

Transformer [24]: A deep learning model that combines recurrent neural networks and attention mechanisms. It focuses on capturing the correlation between previous and subsequent time steps in an input sequence using a self-attentive mechanism.

Informer [39]: An improved Transformer-based model that uses a sparse attention mechanism instead of the traditional attention mechanism and proposes a one-step generative decoding method that significantly improves the speed of long sequence prediction.

Reformer [13]: A Transformer-based variant model that divides the input sequence into multiple buckets, with independent attention computation within each bucket, significantly reducing computational complexity.

DLinear [33]: A deep learning model combining convolutional neural networks and recurrent neural networks. It decomposes a time series into a trend portion and a residual portion and uses a single-layer linear network to predict these two portions separately, improving the accuracy of the prediction.

Table 1
Hyperparameter settings for experiments.

Parameter	Value
batch_size	32
epochs	100
n_heads	8
e_layers	3
d_layers	2
d_model	512
d_ff	2048
learning rate	0.0001
optimizer	Adam

FEDformer [40]: A neural network model based on frequency domain decomposition, which applies global features of the sequence to the frequency domain using Fourier and wavelet transforms, reducing complexity while improving prediction accuracy.

PatchTST [20]: A patch-based Transformer model that reduces the spatial and temporal complexity of the model by treating each patch as an input token that is fed into the Transformer.

4.3. Experimental settings

In order to achieve the best prediction results, we describe the data preprocessing, the model running environment, and the parameter settings in this section.

We first filtered the data for features, eliminating unnecessary features. Then we processed the data for missing values. For numeric missing values, we used averages to fill the missing values in these columns and then updated the filled data back to the original data frame. For date type missing values, the 'Date' column was converted to date format, and the missing date values were filled using forward fill and backward fill methods. With these steps, data integrity is ensured.

In addition, to enhance the robustness of the model in dealing with noisy and biased data, we performed noise and bias processing on the input data. Specifically, we generated a noise tensor for the input data, where each element of the tensor was drawn from a standard normal distribution and multiplied by a preset 5% noise standard deviation to control for noise intensity. Doing so simulates real-world random perturbations and allows the model to learn how to cope with uncertainty in the data. Similarly, we simulated systematic errors in the data by generating a tensor with a standard normal distribution and multiplying it by a 3% standard deviation of bias. After these treatments, the input data contains more complex features that help the model generalize better. Next, we divided the dataset into training, testing, and validation sets in a 7:2:1 ratio to ensure that the model can be trained and tested on different datasets.

We compared six benchmark models – Transformer, Informer, Reformer, DLinear, FEDformer, and PatchTST – with PMformer, as well as with versions of our model with added noise (Ours-n) and added bias (Ours-b), through multivariate and univariate forecasting experiments conducted in the environment with matplotlib == 3.1.1, numpy == 1.19.4, pandas == 0.25.1, scikit_learn == 0.21.3, and torch == 1.8.0. All models were tested under the same parameter settings, which are detailed in Table 1.

To enhance the predictive power of the model and maintain high computational efficiency, the batch size of the model is set to 32 to allow more efficient parallel processing. The number of epochs is set to 100 to ensure that the model is able to adequately learn the temporal dependency patterns in the training data. The number of heads of the attention mechanism is controlled by the n_heads parameter, which is set to 8 to capture multi-scale dependencies in the sequence. The encoder layers (e_layers) and decoder layers (d_layers) are set to 3 and 2, respectively, in order to maintain the computational efficiency and stability of the model while capturing complex time series patterns. The dimension of the hidden layer, d_model, is set to 512 to ensure that the model has sufficient capacity to learn the features of the time series. The dimension of the feedforward network, d_ff, is set to 2048, which ensures that the model has sufficient capacity for nonlinear transformation. In addition, in order to improve the optimization effect, the model uses the Adam optimizer with the learning rate set to 0.0001 to ensure the stability of the training process and the convergence speed.

4.4. Evaluation parameters

In this study, we adopt the widely used mean squared error (MSE) and mean absolute error (MAE) as the main evaluation criteria. These two metrics are commonly used to measure the deviation between the predicted and true values and quantify and compare the performance of different forecasting models on time series forecasting tasks. MAE indicates the average difference between the predicted and true values. Smaller MAE values indicate more accurate forecasts. MAE [5] is shown in equation (15):

$$\mathrm{MAE} = \frac{1}{N}\sum_{k=1}^{N} |y_k - \hat{y}_k| \tag{15}$$

where N denotes the number of samples, y_k denotes the true value, and $\hat{y}_k$ denotes the predicted value. MSE represents the average of the sum of squares of the errors between the predicted and actual values. A smaller MSE value indicates that the predicted value of the model is closer to the actual observed value. MSE [13] is shown in equation (16):

$$\mathrm{MSE} = \frac{1}{N}\sum_{k=1}^{N}(y_k - \hat{y}_k)^2 \tag{16}$$

4.5. Comparative experiment

In both multivariate and univariate time series forecasting tasks, we compared the performance of six benchmark models—Transformer, Informer, Reformer, DLinear, FEDformer, and PatchTST—against PMformer, and calculated the percentage increase in model metrics (comparing optimal results with suboptimal ones). Additionally, we evaluated PMformer's performance in more complex data environments by incorporating noise (Ours-n) and bias (Ours-b) treatments. To simulate randomness and systematic errors in real data, we introduced 5% noise and a 3% offset to the input data. The experimental results are shown in Tables 2 and 3, where the best results are highlighted in bold, and the second-best results are underlined.

As evident from the results presented in Tables 2 and 3, the proposed forecasting model demonstrates strong overall performance in both multivariate and univariate time series forecasting tasks. The model consistently achieves good results across multiple datasets, as indicated by low MSE and MAE scores, for both short-term (24, 48 time steps) and long-term (336, 720 time steps) predictions. This performance holds even when noise and bias are introduced, highlighting the model's capacity to capture temporal dependencies and handle complex data features.

Handling noisy and biased data: The model maintains good predictive accuracy even with the addition of noise (Ours-n) and bias (Ours-b). This robustness stems from PMformer's standardized processing and data enhancement techniques. The model first standardizes input data to zero mean and unit variance, mitigating the impact of noise on training and ensuring that features have similar value ranges. During training, data augmentation techniques are employed, introducing random noise and offsets to the input data. This process simulates realistic data uncertainty, enabling the model to learn to handle noise and bias effectively and enhancing its robustness, prediction performance, and reliability.

Multivariate prediction: The model excels in both short- and long-term multivariate prediction tasks due to its effective handling of complex data. PMformer receives multivariate input data, often comprising various features, which are first reshaped and normalized. After handling missing values, relevant feature columns are selected and arranged based on specified column names, excluding the target and date columns. The data are then divided into training, validation, and testing sets to ensure a balanced distribution. The data normalization process involves calculating the mean and standard deviation of the training data. These statistics are then applied to the entire dataset, transforming each feature column to a normalized form with zero mean and unit variance. This normalization step minimizes the influence of differing scales across features, contributing to more stable model training.

In short-term multivariate prediction (24, 48 time steps), the model outperforms Reformer and DLinear. This advantage arises from its probabilistic patch sampling mechanism, which efficiently focuses on critical short-term information by reducing computation on less relevant long-term dependencies. Consequently, the model achieves lower error rates in short-term multivariate forecasting. The model demonstrates even stronger performance in long-term prediction (336, 720 time steps). This superior performance is attributed to the incorporation of the multi-scale scaling sparse attention technique, which enhances the model's ability to capture global temporal dependencies. By dynamically adjusting the attention distribution across different scales, the model can more accurately predict future trends, particularly when dealing with long time series. Moreover, compared to Informer and Transformer, PMformer demonstrates greater stability in handling complex multivariate dependencies. This stability is particularly evident in long-term prediction, where the model exhibits reduced fluctuations and error accumulation.

Univariate prediction: The model also shows good performance in univariate short time-step prediction. The model optimizes the processing of historical information in the time series by dilated causal pooling layer, which makes it possible to effectively capture the impact of historical states on future states even in the case of short time steps, thus providing more accurate prediction results than other models. The model further comes into its own in univariate long time-step prediction. The use of a multilayer perceptual cross self-attentive decoder improves the model's ability to capture deep features of a single time series. This allows the model to maintain a high level of accuracy and a low error rate when making predictions over days or even weeks.

Performance improvement testing: To assess the statistical significance of each metric, we calculated the p-values for each metric using the Wilcoxon test. The computed p-values are all less than 0.05, indicating that the performance differences between PMformer and the other models are statistically significant. Moreover, the last two columns of the tables demonstrate that PMformer has achieved performance improvements over the suboptimal model in both metrics, further highlighting the significance of PMformer's effectiveness.

This shows that the model proposed in this paper not only performs well in short time-step forecasting but is also applicable to long-term series forecasting by designing an advanced attention mechanism and optimized network structure. In particular, the model demonstrates significant advantages when dealing with multivariate and univariate time series data with complex dependency structures.

4.6. Ablation experiments

To analytically validate the effectiveness of each module of PMformer, we conducted multivariate and univariate ablation experiments on four datasets. "Ours" denotes the full model, "-PPSA" denotes that the probabilistic patch sampling attention mechanism was not used, and "-MSSA" denotes that the multi-scale scaling sparse attention mechanism was not used. The multivariate prediction results are shown in Figs. 4–7, and the univariate prediction results are shown in Figs. 8–11.

Table 2
Multivariate prediction results of the models on 6 datasets.

	Metric	Ours		Ours-n		Ours-b		PatchTST		FEDformer		DLinear		Reformer		Informer		Transformer		Impr.	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
NFLX	24	**0.782**	0.555	0.783	0.563	0.792	**0.533**	0.857	0.590	0.942	0.617	0.852	0.595	1.605	0.949	1.654	0.953	1.612	0.944	8.2%	5.9%
	48	**0.808**	**0.549**	0.824	0.584	0.909	0.576	0.997	0.663	1.329	0.778	1.054	0.716	1.493	0.920	1.563	0.932	1.450	0.899	19.0%	17.2%
	96	**0.917**	**0.651**	1.010	0.710	0.935	0.665	1.278	0.784	1.642	0.886	1.288	0.822	1.465	0.926	1.436	0.890	1.336	0.872	28.2%	17.0%
	168	**0.907**	**0.665**	0.939	0.692	0.943	**0.665**	1.668	0.999	2.038	1.083	1.332	0.891	1.413	0.936	1.319	0.886	1.089	0.783	16.7%	15.1%
	216	**0.898**	**0.693**	0.938	0.707	0.977	0.725	1.824	1.073	2.458	1.249	1.217	0.850	1.373	0.915	1.300	0.879	1.033	0.759	13.1%	8.7%
Seattle-weather	24	**0.615**	**0.486**	0.670	0.519	0.640	0.497	0.712	0.546	0.770	0.601	0.840	0.618	0.746	0.594	0.702	0.532	0.693	0.525	11.3%	7.4%
	48	**0.527**	**0.474**	0.581	0.497	0.554	0.482	0.842	0.620	0.915	0.623	0.960	0.680	0.813	0.646	0.712	0.532	0.676	0.527	22.0%	10.1%
	96	**0.555**	**0.507**	0.655	0.554	0.606	0.528	1.087	0.733	1.250	0.812	1.198	0.780	0.967	0.743	0.777	0.639	0.852	0.672	28.6%	20.7%
	168	**0.787**	**0.645**	0.898	0.733	0.900	0.712	1.201	0.828	1.634	1.013	1.362	0.907	1.150	0.836	0.928	0.746	1.030	0.781	15.2%	13.5%
	216	**1.013**	**0.751**	1.058	0.812	1.090	0.792	1.143	0.785	1.606	0.995	1.165	0.836	1.390	0.910	1.418	0.913	1.250	0.850	11.4%	4.3%
Solarenergy	24	**0.800**	**0.694**	0.872	0.732	0.877	0.725	1.053	0.793	1.279	0.886	1.066	0.792	1.017	0.798	1.011	0.797	0.850	0.727	5.9%	4.5%
	36	**1.086**	**0.838**	1.118	0.842	1.108	0.847	1.239	0.851	1.303	0.897	1.227	0.857	1.262	0.886	1.133	0.859	1.107	0.852	1.9%	1.5%
	48	**1.094**	**0.829**	1.112	0.843	1.145	0.858	1.190	0.831	1.442	0.943	1.203	0.845	1.275	0.894	1.436	0.989	1.295	0.935	8.1%	0.2%
	60	**1.077**	**0.805**	1.144	0.845	1.117	0.832	1.166	0.827	1.274	0.876	1.223	0.849	1.289	0.897	1.293	0.926	1.192	0.880	7.6%	2.7%
	72	**1.085**	**0.822**	1.126	0.831	1.087	**0.822**	1.195	0.837	1.234	0.883	1.224	0.852	1.251	0.883	1.404	0.967	1.132	0.837	4.2%	1.8%
Traffic_volume	24	**0.339**	**0.409**	0.347	0.421	0.340	0.421	0.557	0.537	0.423	0.458	0.623	0.622	0.366	0.453	0.459	0.520	0.353	0.414	4.0%	1.2%
	48	**0.390**	0.450	0.396	0.447	0.397	**0.442**	0.640	0.619	0.432	0.491	0.684	0.663	0.462	0.542	0.581	0.600	0.416	0.496	6.2%	8.4%
	168	0.515	**0.519**	0.549	0.588	**0.513**	0.521	0.740	0.713	0.518	0.554	0.756	0.718	0.730	0.729	0.747	0.720	0.717	0.710	0.6%	6.3%
	336	**0.514**	**0.520**	0.519	0.553	0.533	0.540	0.764	0.733	0.520	0.577	0.784	0.741	0.760	0.750	0.801	0.750	0.739	0.720	1.2%	27.8%
	720	**0.510**	**0.545**	0.548	0.558	0.537	0.561	0.811	0.763	0.538	0.587	0.819	0.767	0.552	0.604	0.837	0.761	0.564	0.577	5.2%	5.5%
Pox	12	**0.718**	**0.667**	0.731	0.681	0.736	0.675	1.031	0.745	1.205	0.843	0.848	0.691	0.813	0.720	1.006	0.806	0.866	0.735	11.7%	3.5%
	18	0.781	**0.699**	**0.776**	0.701	0.820	0.734	1.050	0.769	1.374	0.910	1.015	0.772	0.808	0.717	0.985	0.806	0.916	0.763	3.3%	2.5%
	24	**0.702**	**0.641**	0.755	0.674	0.721	0.655	1.167	0.817	1.371	0.904	1.023	0.777	0.825	0.729	0.970	0.804	0.891	0.750	14.9%	12.1%
	30	**0.654**	**0.605**	0.734	0.658	0.680	0.627	1.107	0.786	1.217	0.845	0.910	0.711	0.829	0.731	0.947	0.791	0.940	0.776	21.1%	14.9%
	36	**0.628**	**0.586**	0.672	0.629	0.680	0.614	0.960	0.706	1.082	0.778	0.824	0.668	0.846	0.736	0.927	0.773	0.937	0.768	23.8%	12.3%
Departure	24	**0.261**	**0.349**	0.269	0.353	0.274	0.357	0.345	0.406	0.418	0.477	0.424	0.479	0.302	0.370	0.275	0.365	0.322	0.405	5.1%	4.4%
	48	**0.302**	**0.381**	0.317	0.390	0319	0.399	0.468	0.476	0.698	0.634	0.431	0.479	0.331	0.409	0.326	0.403	0.408	0.462	7.4%	5.5%
	168	0.401	**0.435**	0.418	0.486	**0.380**	0.439	0.538	0.538	1.347	0.925	0.582	0.540	0.455	0.487	0.514	0.532	0.650	0.604	11.9%	10.7%
	336	0.395	**0.431**	0.397	0.452	**0.365**	0.434	0.464	0.498	0.855	0.741	0.442	0.467	0.421	0.471	0.654	0.613	0.779	0.683	6.2%	7.7%
	720	**0.365**	0.427	0.394	0.442	0.369	**0.423**	0.550	0.549	0.926	0.767	0.522	0.518	0.419	0.467	0.779	0.681	0.824	0.714	12.9%	8.6%

Table 3
Univariate prediction results of the models on 6 datasets.

	Metric	Ours		Ours-n		Ours-b		PatchTST		FEDformer		DLinear		Reformer		Informer		Transformer		Impr.	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
NFLX	24	**0.148**	**0.275**	0.160	0.284	0.153	0.280	0.181	0.285	0.222	0.323	0.224	0.343	0.170	0.297	0.180	0.304	0.211	0.325	12.9%	3.5%
	48	**0.209**	**0.319**	0.219	0.336	0.238	0.343	0.266	0.352	0.533	0.513	0.455	0.502	0.272	0.396	0.238	0.337	0.330	0.398	12.2%	5.3%
	96	**0.335**	0.439	**0.335**	0.449	0.355	**0.434**	0.798	0.594	1.309	0.803	1.131	0.789	0.387	0.516	0.346	0.441	0.540	0.537	3.2%	0.5%
	168	**0.296**	**0.446**	0.308	0.477	0.359	0.488	2.785	1.282	2.108	1.136	1.384	0.974	0.312	0.453	0.406	0.556	0.543	0.610	5.1%	1.5%
	216	**0.311**	**0.480**	0.341	0.481	0.337	0.488	3.754	1.638	2.410	1.280	1.271	0.945	0.389	0.508	0.362	0.515	0.590	0.646	14.1%	5.5%
Seattle-weather	24	**0.696**	0.638	0.698	0.653	0.699	**0.625**	0.783	0.665	0.762	0.659	0.884	0.724	0.798	0.661	0.819	0.663	0.793	0.659	8.7%	3.2%
	48	**0.634**	**0.618**	0.737	0.625	0.738	0.623	0.763	0.648	0.786	0.643	0.894	0.726	0.732	0.632	0.796	0.643	0.739	0.619	13.4%	0.2%
	96	**0.634**	**0.594**	0.646	0.597	0.704	0.595	0.909	0.729	0.722	0.603	0.839	0.714	0.675	**0.594**	0.834	0.656	0.790	0.623	6.1%	0.0%
	168	**0.806**	**0.698**	0.809	**0.698**	0.822	0.704	0.988	0.781	0.930	0.758	0.816	0.710	1.231	0.938	0.921	0.765	0.987	0.813	1.2%	1.7%
	216	**0.826**	**0.719**	0.831	0.734	0.945	0.732	1.067	0.826	0.979	0.791	1.052	0.817	1.318	0.872	0.914	0.764	1.066	0.778	9.6%	5.9%
Solarenergy	24	**0.163**	**0.299**	0.185	0.317	0.184	0.319	0.230	0.367	0.254	0.394	0.848	0.774	0.352	0.468	0.425	0.520	0.357	0.473	29.1%	18.5%
	36	**0.263**	**0.399**	0.285	0.418	0.284	0.417	0.307	0.429	0.347	0.470	0.847	0.755	0.474	0.549	0.524	0.572	0.545	0.586	14.3%	7.0%
	48	**0.349**	**0.467**	0.351	0.470	0.363	0.477	0.366	0.475	0.363	0.484	0.663	0.653	0.515	0.573	0.588	0.624	0.526	0.579	3.9%	1.7%
	60	0.299	0.428	0.360	0.472	**0.290**	**0.420**	0.349	0.449	0.366	0.482	0.579	0.605	0.531	0.589	0.633	0.651	0.559	0.600	14.3%	4.7%
	72	0.385	0.489	0.382	**0.488**	**0.380**	0.491	0.402	0.496	0.387	0.492	0.506	0.558	0.553	0.597	0.913	0.790	0.528	0.584	0.5%	0.6%
Traffic_volume	24	0.184	0.345	0.212	0.364	**0.150**	**0.295**	0.516	0.531	0.296	0.405	0.713	0.693	0.275	0.408	0.357	0.446	0.599	0.640	33.1%	14.8%
	48	**0.169**	**0.332**	0.219	0.380	0.188	0.347	0.716	0.683	0.286	0.399	0.799	0.747	0.275	0.394	0.406	0.496	0.637	0.676	38.5%	15.7%
	168	**0.172**	0.332	0.176	**0.328**	0.240	0.389	0.935	0.823	0.381	0.468	0.922	0.824	0.261	0.388	0.397	0.492	0.501	0.577	34.1%	14.4%
	336	**0.158**	**0.309**	0.166	0.321	0.172	0.321	0.976	0.853	0.398	0.479	0.962	0.847	0.259	0.387	0.674	0.688	0.604	0.648	39.0%	20.2%
	720	0.308	**0.391**	0.352	0.497	**0.298**	0.430	1.023	0.872	0.386	0.468	0.987	0.862	0.347	0.489	0.968	0.862	1.002	0.813	11.2%	16.5%
Pox	12	**0.433**	**0.506**	0.445	0.514	0.460	0.523	0.806	0.651	0.834	0.654	1.077	0.807	0.535	0.549	0.652	0.617	0.696	0.641	19.1%	7.8%
	18	**0.463**	**0.506**	0.506	0.549	0.485	0.528	0.956	0.717	1.003	0.756	1.346	0.937	0.573	0.568	0.657	0.639	0.728	0.679	19.2%	10.9%
	24	**0.547**	**0.561**	0.558	0.565	0.579	0.591	0.843	0.665	0.952	0.738	1.375	0.959	0.647	0.637	0.627	0.615	0.603	0.608	9.3%	7.7%
	30	**0.499**	**0.530**	0.519	0.551	0.533	0.545	0.582	0.532	0.680	0.606	1.192	0.872	0.691	0.663	0.615	0.607	0.625	0.617	14.3%	0.4%
	36	**0.517**	**0.539**	0.521	0.540	0.521	0.540	0.601	0.572	0.569	0.548	1.013	0.802	0.699	0.669	0.642	0.614	0.678	0.654	9.1%	1.6%
Departure	24	**0.477**	0.486	0.499	0.486	0.478	**0.485**	0.501	0.495	0.552	0.576	0.514	0.503	0.498	0.487	0.558	0.517	0.687	0.638	4.2%	0.2%
	48	**0.539**	**0.516**	0.561	0.522	0.556	0.529	0.614	0.538	0.627	0.616	0.598	0.544	0.594	0.533	0.571	0.544	0.580	0.526	5.6%	1.9%
	168	**0.604**	**0.570**	0.646	0.580	0.647	0.587	0.640	0.579	0.666	0.611	0.638	0.585	0.662	0.582	0.762	0.633	0.695	0.600	5.3%	1.6%
	336	**0.605**	**0.571**	0.670	0.592	0.680	0.595	0.606	0.579	0.673	0.604	0.644	0.586	0.636	0.585	0.709	0.612	0.665	0.572	0.2%	0.2%
	720	0.676	0.599	0.659	**0.582**	**0.625**	**0.582**	0.676	0.603	0.737	0.626	0.717	0.611	0.719	0.628	0.828	0.685	0.753	0.656	0.0%	0.7%

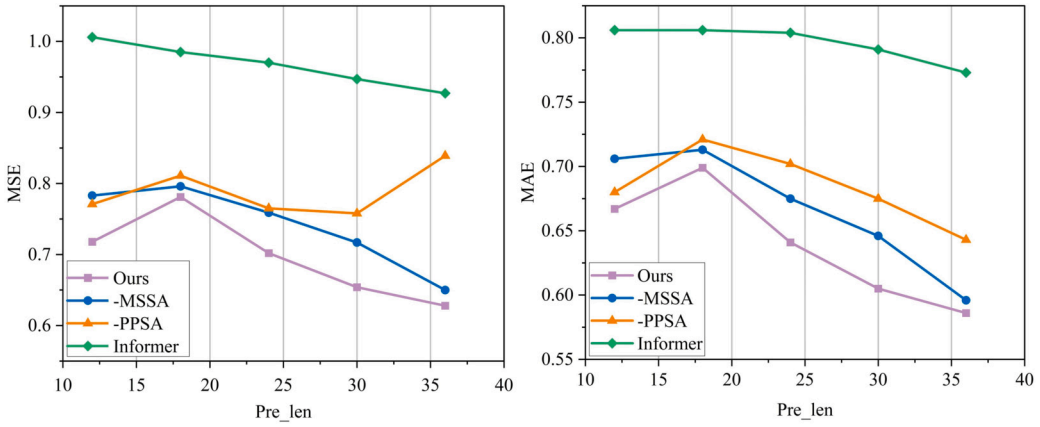

Fig. 4. Multivariate ablation study of the PMformer components on the Pox dataset.

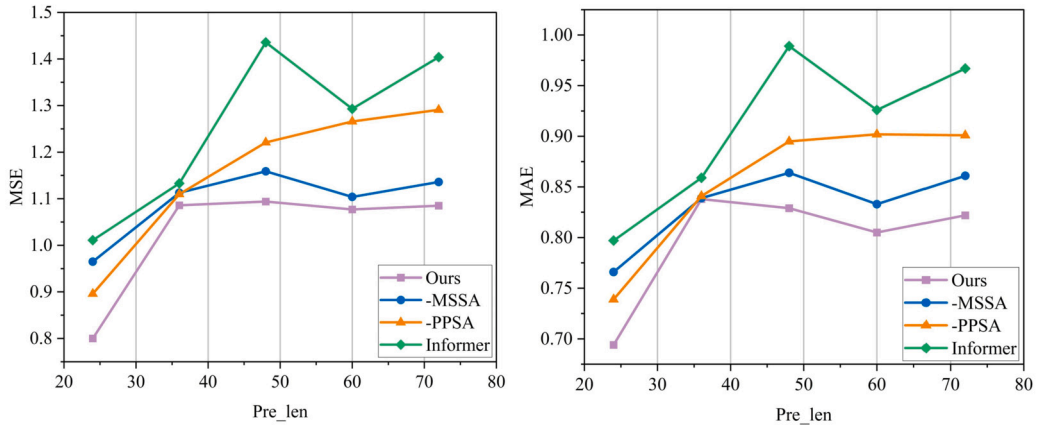

Fig. 5. Multivariate ablation study of the PMformer components on the Solarenergy dataset.

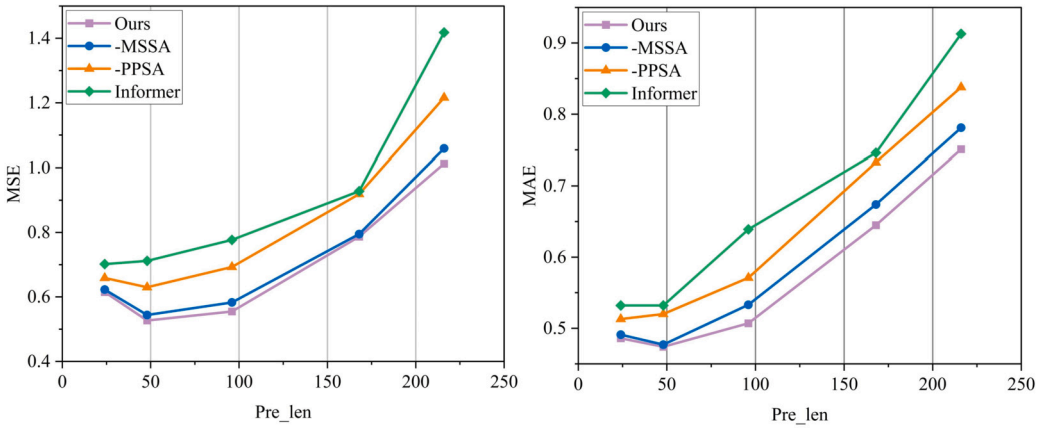

Fig. 6. Multivariate ablation study of the PMformer components on the Seattle-weather dataset.

As can be seen in Figs. 4–7, the complete PMformer model overall outperforms the variant models with key components removed in terms of multivariate prediction. The full model demonstrated lower MSE values on all four datasets, especially in the long prediction range. Fig. 7 shows the significant advantage of the full model on the long prediction, where the MSE of the full model is much lower than the model with the MSSA or PPSA components removed for the 720-step prediction. The variants with the MSSA and PPSA components removed exhibited higher MSE, suggesting that these components are critical for reducing prediction errors, especially when the prediction range is large. For MAE, the full model similarly demonstrated more stable and accurate results across all datasets and prediction lengths. As can be seen in Fig. 4, the MAE of the full model is significantly lower than that of the model

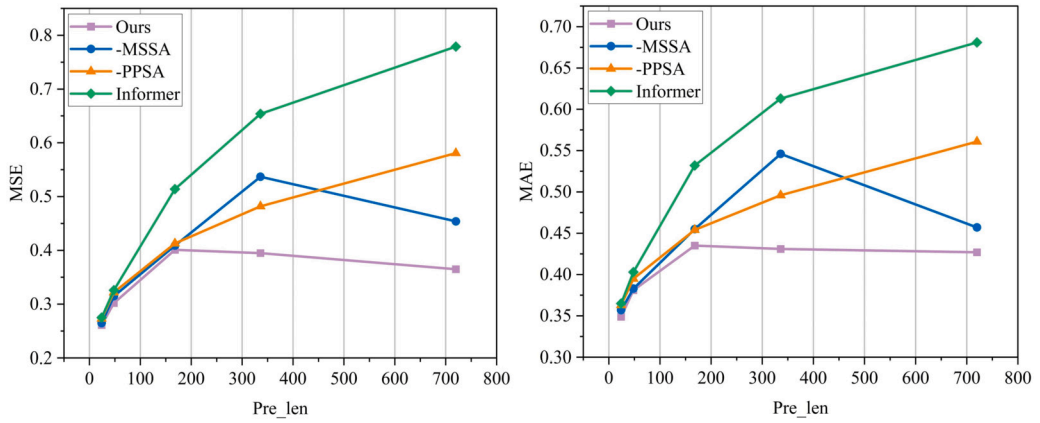

Fig. 7. Multivariate ablation study of the PMformer components on the Departure dataset.

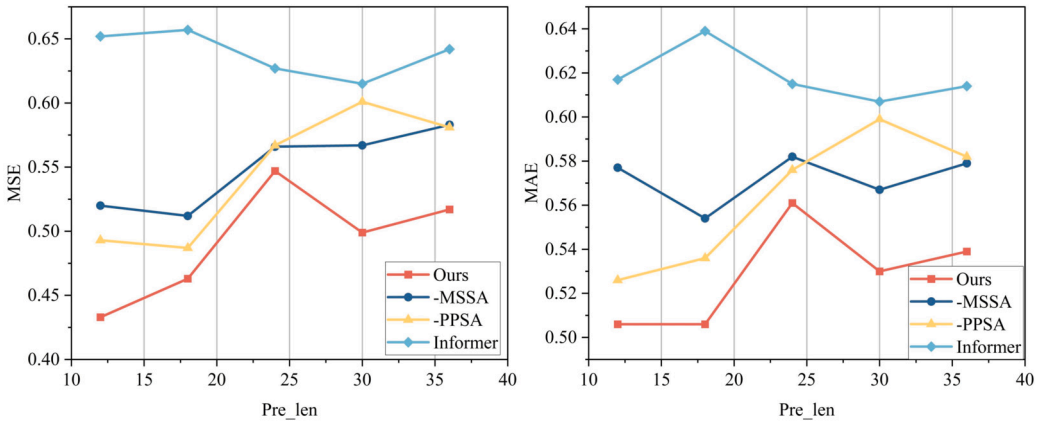

Fig. 8. Univariate ablation study of the PMformer components on the Pox dataset.

with one component missing, even in short time-step predictions. This suggests that the full model provides more accurate predictive values even in short-term predictions with fast responses. The MAE value of the full model remains low in the long-term prediction task, showing its strength in maintaining long-term prediction accuracy. The full model effectively reduces computational inputs for unnecessary time patches through the probabilistic patch sampling attention mechanism, allowing the model to focus its resources on time patches that have a decisive impact. The multi-scale scaling sparse attention mechanism, on the other hand, dynamically adjusts attention on multiple time scales, effectively capturing long-term dependencies while maintaining sensitivity to short-term events. Models that remove the PPSA or MSSA components lack sufficient flexibility and sensitivity when dealing with multivariate time series. Models without a probabilistic patch sampling mechanism waste computational resources on unimportant data patches, while models lacking a multi-scale scaling sparse attention mechanism have difficulty managing and exploiting both long- and short-term dependencies of the time series, resulting in lower prediction accuracy.

The full PMformer model similarly exhibits superior performance over the other variants for univariate forecasting. Figs. 8–11 present the results of the model for various prediction time intervals from the short to the long term, showing the model's adaptability and accuracy for different lengths of predictions. For MSE, the prediction results for the Pox dataset show that the full model significantly outperforms the variant model with missing key components over short time spans of 20 to 40 steps. This efficient short-term forecasting ability is attributed to the model's ability to quickly adapt and accurately capture short-term fluctuations in the time series. On the Departure dataset 720-step prediction, the MSE of the full model is similarly lower than the other variants, which shows the stability of the model when dealing with long sequence data. For MAE, the full model likewise exhibits a smaller deviation from the true value, which indicates the accuracy of its results. On the Seattle-weather dataset, the full model consistently demonstrates smaller MAE values than the other variants for forecasts from 50 to 250 steps. This reflects the model's ability to accurately reflect immediate changes by closely following actual data trends in the short term. On the Departure dataset, the MAE of the full model stays low for long-term forecasts from 300 to 700 steps, which highlights the stability and accuracy of the model in tracking and forecasting time series over consistently long periods. This performance is attributed to the probabilistic patch sampling mechanism that concentrates on data at key moments, and the multi-scale sparse attention that allows the model to balance its attention across different time scales, effectively capturing the dynamics from short-term fluctuations to long-term trends, and ensuring the accuracy of forecasts. In model variants without PPSA or MSSA, the model loses its ability to respond quickly to unexpected events in the time

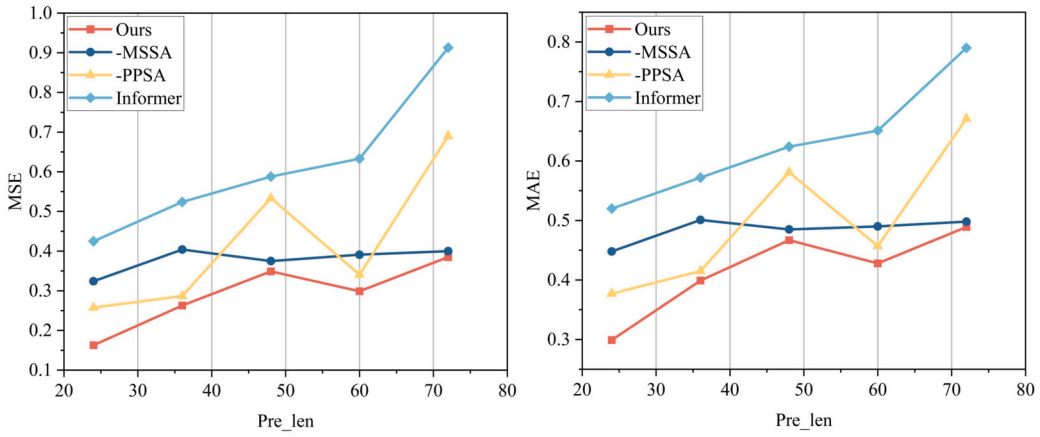

Fig. 9. Univariate ablation study of the PMformer components on the Solarenergy dataset.

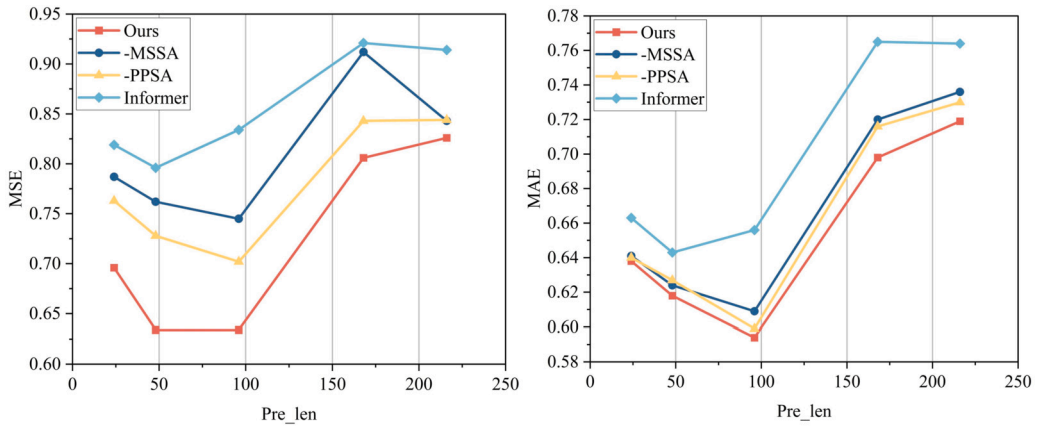

Fig. 10. Univariate ablation study of the PMformer components on the Seattle-weather dataset.

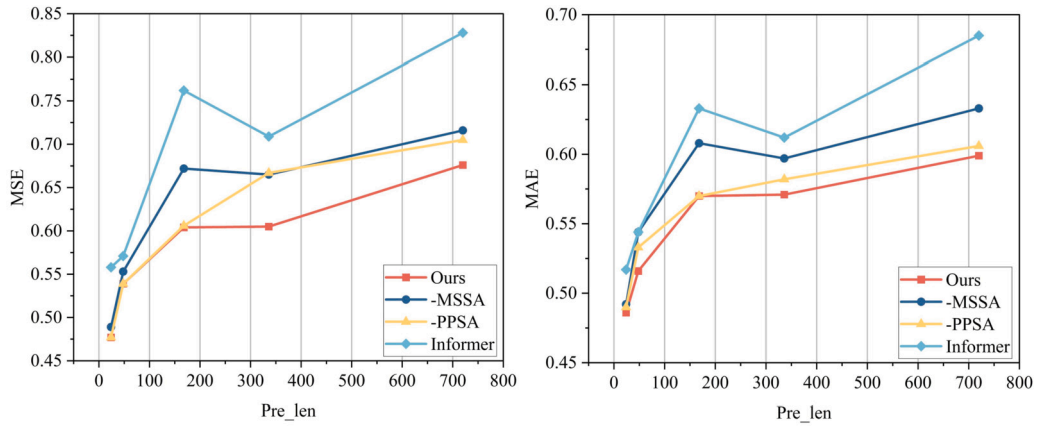

Fig. 11. Univariate ablation study of the PMformer components on the Departure dataset.

series and also lacks the ability to consistently track and forecast long-term trends. This results in a model that is prone to larger errors as the forecasting time horizon increases.

4.7. Parameter sensitivity

To assess the impact of different parameters in the PPSA mechanism on model performance, we performed parameter sensitivity analysis on three datasets. **Pruning rate:** We tested different pruning rate settings, including 0.1, 0.3, 0.5, 0.7, 0.9, to observe its

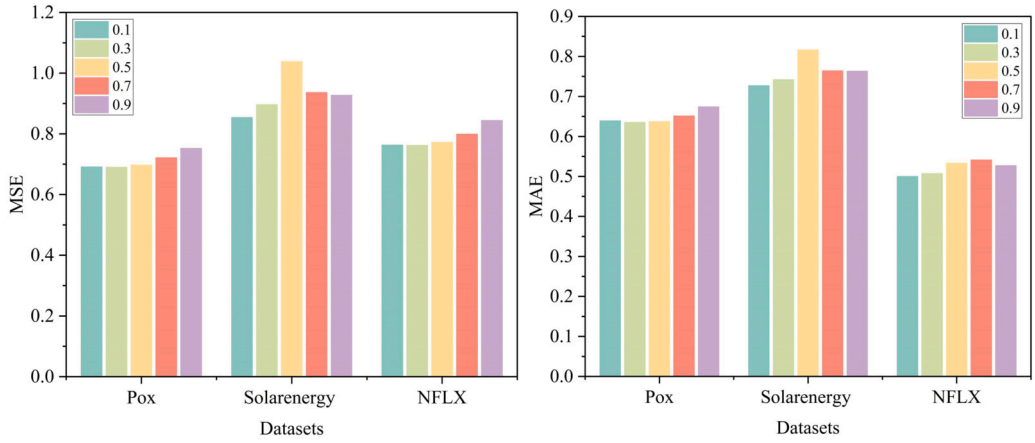

Fig. 12. Impact of pruning rate on model performance.

effect on model performance, and the results are shown in Fig. 12. **Number of patches:** We set the number of selected patches to 4, 8, 16, 32, 64 to analyze the impact of different numbers of patches on model performance, and the results are shown in Fig. 13. **Patch size:** We adjusted the length of the patches, including 5, 10, 15, 20, 25, to explore the specific effect of patch size on model performance, and the results are shown in Fig. 14.

Fig. 12 demonstrates the effects of different pruning ratios on model performance for the three datasets, Pox, Solarenergy, and NFLX, in terms of the two metrics MSE and MAE. Overall, the effects of pruning ratios on these datasets vary, indicating that the choice of pruning strategy needs to be tailored to specific data characteristics.

On the Pox dataset, the MSE reaches its lowest point at a pruning rate of 0.3 and then gradually increases as the pruning rate increases. This indicates that, on the Pox dataset, moderate pruning helps reduce model complexity without sacrificing too much useful information. The MAE is similarly lowest at a pruning rate of 0.3 and then continues to rise as the pruning rate increases. This is because a lower pruning rate retains more detailed information that is favorable for prediction, while a higher pruning rate may oversimplify the model and lose critical information. Therefore, for the Pox dataset, the pruning rate is most appropriately set at 0.1–0.3.

On the Solarenergy dataset, MSE is lowest at a pruning rate of 0.1 and shows an overall increasing trend as the pruning rate increases, especially peaking at 0.5. This suggests that a lower pruning rate is more effective for the Solarenergy dataset because the dataset contains more complex patterns that need to be predicted by fine-grained information. The trend for MAE is similar to MSE in that it is also lowest at a pruning ratio of 0.1 and increases as the pruning rate increases. This further emphasizes that a lower pruning rate on this dataset better captures and predicts subtle changes in the data. Therefore, for the Solarenergy dataset, we set the pruning rate near 0.1.

On the NFLX dataset, it can be seen from the figure that MSE is lowest at a pruning rate of 0.3, and as the pruning rate increases (0.3, 0.5, 0.7, 0.9), MSE gradually increases. This suggests that a lower pruning rate is more effective on the NFLX dataset because a lower pruning rate retains more useful information and thus reduces the error. The trend of MAE on the NFLX dataset is similar to that of MSE. The MAE is lowest at a pruning rate of 0.1 and gradually increases as the pruning rate increases. This indicates that as the pruning rate increases, the model discards some key information elements, which negatively affects the prediction results. While increasing the pruning rate can simplify the model and speed up computation, in this case it may weaken the model's ability to capture and interpret complex patterns in the data. Therefore, for the NFLX dataset, we set the pruning rate near 0.2.

In summary, when applying the pruning strategy, choosing the appropriate pruning rate is crucial and requires a comprehensive consideration of the balance between reducing the computational burden and maintaining model performance.

Fig. 13 demonstrates the effect of different numbers of patches on the three datasets of Pox, Solarenergy, and NFLX in terms of model performance. It can be seen that the variation in the number of patches significantly impacts different datasets, indicating that appropriately choosing the number of patches plays a key role in optimizing model performance.

The MSE for the Pox dataset is lowest when the number of patches is 32, indicating that a larger number of patches helps capture the cyclical variation in this dataset. When the number of patches increases to 64, the MSE increases slightly, which is due to excessive model complexity. The trend for MAE is similar to MSE and also reaches its lowest at a patch number of 32. This indicates that the model predicts the actual values more accurately at this patch setting and that a higher number of patches did not result in a performance improvement. Therefore, we set the number of patches to around 32.

On the Solarenergy dataset, the MSE reaches its lowest at a patch number of 16, indicating that this setting provides the best feature extraction and information balance on this dataset. As the number of patches increased to 64, the MSE increased, as too many patches introduced noise or caused overfitting. MAE is also lowest at a patch number of 16, which is consistent with MSE's performance. The MAE is higher at lower (4 and 8) or higher (32 and 64) numbers of patches, suggesting that the model fails to capture key trends or information in the data effectively at these settings. Fewer patches are enough to capture critical information, while more patches lead to unnecessary complexity and noise. Therefore, we set the number of patches near 16.

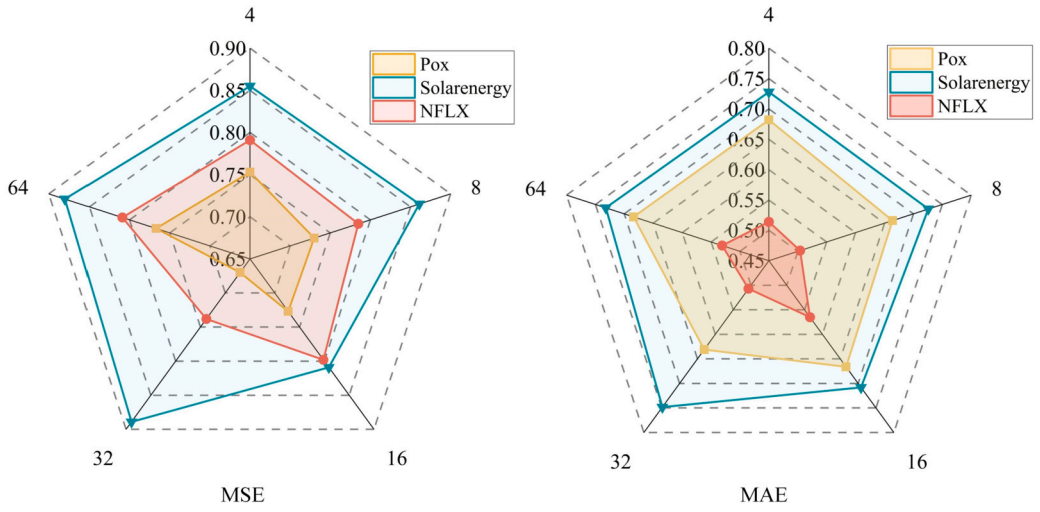

Fig. 13. Effect of the number of patches on model performance.

In the NFLX dataset, with fewer patch settings (such as 4 and 16), the model performs poorly in capturing key information and managing the overall error, especially when there are only 4 patches, and the model performance decreases dramatically. However, in the 32-patch configuration, the MSE is significantly lower, and although the MAE rises compared to the 8-patch, this trade-off is acceptable in terms of overall error management, suggesting that this configuration provides an optimal balance between accuracy and efficiency. On the contrary, more patches (64) did not lead to performance improvement but rather to a slight performance degradation due to the over-complexity of the model. Therefore, for the NFLX dataset, we chose to set the number of patches around 32.

To summarize, different datasets have different needs for the number of patches. Choosing the appropriate number of patches can significantly affect the model's predictive performance. Too many or too few patches may lead to performance degradation. Therefore, the number of patches needs to be adjusted for the specific dataset during model design.

Fig. 14 illustrates the effect of different patch sizes on MSE and MAE for the three datasets. The figures show that patch size significantly affects model performance, and different datasets have different sensitivities to patch size.

For the Pox dataset, the MSE reaches its lowest at a patch size of 10, showing that this size is best suited to capture the critical features of the dataset, effectively reducing the prediction error. However, when the patch size is increased to 20, the MSE is significantly higher. This is due to the performance degradation caused by too-large patches capturing too much irrelevant information or not being able to effectively differentiate details in the data. At the same time, MAE decreases with increasing patch size, indicating that larger patch sizes, while more stable in single-point prediction, are less effective in predicting trends in aggregate data. Therefore, we set the patch size to around 10.

For the Solarenergy dataset, the MSE reaches its lowest at a patch size of 10, suggesting that this smaller patch size is best suited to accurately capture variations in this dataset, especially short-term fluctuations, and can maximize model performance. The MSE increases significantly when the patch size is increased to 25. This is because larger patches lose sensitivity to key local features when attempting to cover a wider range of data, resulting in the model not being able to effectively differentiate between specific changes over time. Meanwhile, the performance of MAE is also optimal at a patch size of 10, after which it gradually increases as the patch size increases. This suggests that smaller patch sizes provide high sensitivity to short-term internal movements, which is crucial for forecasting accuracy. Therefore, we set the patch size to around 10.

On the NFLX dataset, there is a slight increase in MSE when the patch size goes from 5 to 10, and this slight increase suggests that the model cannot capture enough contextual information at very small patch sizes, while slightly larger patch sizes fail to improve model performance significantly. The MAE shows the same trend, which further suggests that the model's prediction accuracy for individual data points is not very high with these smaller patch sizes, suggesting that these smaller patches are too limited to capture key movements in the data effectively. Both MSE and MAE perform optimally with a patch size of 15, which is because this patch size provides enough information while avoiding too much noise, effectively balancing the relationship between capturing key data features and avoiding overfitting. Patch sizes of 20 and 25 show an increasing and then decreasing trend in both MSE and MAE, and although this indicates an improvement in the model's single-point prediction accuracy, overall, larger patch sizes do not result in sustained performance gains. Therefore, we set the patch size near 15.

In summary, proper patch size selection is key to model optimization and needs to be carefully tuned to the data characteristics and prediction needs to ensure the model can effectively capture and utilize key information from the data.

4.8. Random perturbation evaluation experiment

In order to verify whether the noise and bias introduced by the model can realistically simulate the data variations that may occur in reality, and thus to evaluate the working performance of the model in different environments, we conducted random perturbation

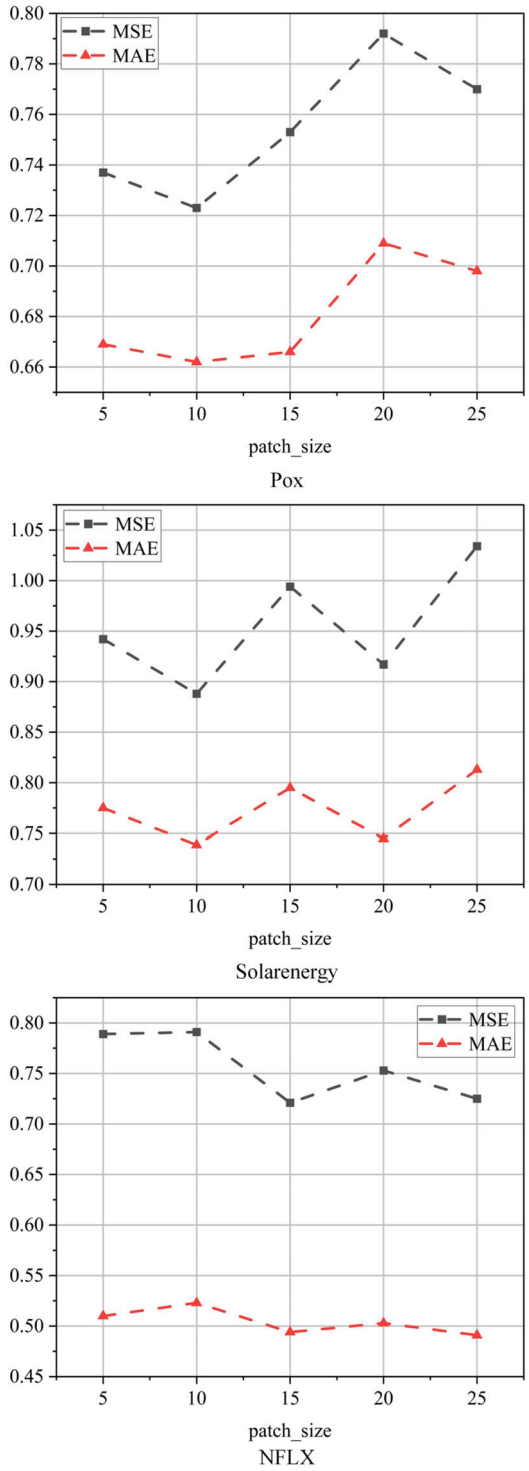

Fig. 14. Influence of patch size on model performance.

evaluation experiments on three datasets, namely, NFLX, Seattle-weather, and Solarenergy. We performed statistical characterization of the data with the added random perturbation against the actual data to observe the consistency of the mean and variance. If the data after adding the perturbations maintain statistical properties very close to the original data, this indicates that these random perturbations are somewhat indistinguishable from the inherent randomness of real-world data. The experimental results are shown in Table 4.

Table 4
Random perturbation evaluation experiment.

Datasets	NFLX		Seattle-weather		Solarenergy	
	Mean	Variance	Mean	Variance	Mean	Variance
Original data	-0.0307	0.9035	0.0094	1.0056	-0.0040	0.9968
Noisy data	-0.0310	0.9059	0.0094	1.0085	-0.0038	0.9987
Biased data	-0.0307	0.9068	0.0090	1.0089	-0.0041	0.9993

Table 5
Analysis of time complexity.

Attention mechanism	Time complexity
ProbSparse Self-attention	$O(B \cdot H \cdot L_Q \cdot (c \cdot \ln(L_K) + L_K))$
Self-attention	$O(B \cdot H \cdot L^2)$
Probabilistic patch sampling attention	$O(B \cdot H \cdot \text{num\_patches} \cdot \text{patch\_size}^2)$

As can be seen from Table 4, the changes in the mean and variance on the three data sets are very small, and the original characteristics of the data are maintained. The means of the original data and the data after the addition of noise and bias on the three data sets are very close to each other. This indicates that these perturbations did not significantly change the center of the data, and the mean values remained stable. Also, the change in variance was minimal. Although the perturbations increased the variance slightly, the change was small. This indicates that the perturbations did not significantly increase the dispersion of the data and maintained the original characteristics of the data. From the comparison of the mean and variance, the random perturbations added did not significantly change the overall characteristics of the data and maintained the statistical characteristics very close to the original data. This suggests that these random perturbations are somewhat indistinguishable from the inherent randomness of real-world data. In other words, the model can better simulate and adapt to the randomness in the real world when dealing with these perturbed data, enhancing its robustness and generalization ability.

The model deals with randomness in the data by introducing random perturbations to enhance its ability to adapt to real-world data. Specifically, the model adds random noise and bias to the input data during training. First, random noise with the same shape as the input data is used and multiplied by the noise standard deviation, which is added to the original data to generate noisy data. Subsequently, additional bias is further added to the noisy data to generate data with noise and bias. The model evaluates the effect of these perturbations on the statistical properties of the data by calculating and accumulating the mean and variance of the raw, noisy, and biased data. This approach simulates the randomness of the data in the real world and helps the model learn to deal with this randomness during the training process, thus improving the robustness and generalization of the model.

4.9. Analysis of time complexity

To thoroughly verify that our proposed probabilistic patch sampling attention mechanism reduces computational complexity, we compared its time complexity with that of the commonly used ProbSparse Self-attention and Self-attention mechanisms. The results are presented in Table 5.

In Table 5, B denotes the batch size, H denotes the number of heads, L_Q denotes the length of queries, c denotes the factor multiplier, L_K denotes the length of keys, num_patches represents the number of sampled patches, patch_size denotes the size of each patch, and L is the sequence length.

As shown in Table 5, the complexity of Self-attention is proportional to the square of the sequence length. This means that for longer input sequences, the computational cost increases rapidly, leading to lower efficiency during both training and inference. Particularly when dealing with large datasets, Self-attention may consume substantial computational resources and memory, making it impractical for long sequence tasks. ProbSparse Self-attention reduces the computational load by introducing a logarithmic factor, making it more efficient than Self-attention. It achieves this by reducing the number of keys to be computed through random sampling, thus avoiding the direct computation of interactions between each query and all keys. While still influenced by the length of queries and keys, its complexity grows significantly slower, making it more suitable for medium-length sequences. The probabilistic patch sampling attention mechanism, by restricting attention computation to a fixed number of local patches, results in its complexity largely depending on the number and size of the patches. This design makes it particularly efficient for long sequences, as it significantly reduces the number of attention pairs that need to be computed, thereby minimizing computational resource consumption during training and inference.

In summary, Self-attention exhibits the highest complexity, resulting in high overhead for long sequence processing and making it unsuitable for resource-constrained environments. The probabilistic patch sampling attention mechanism demonstrates the lowest complexity, making it most efficient for long sequence applications and suitable for large-scale datasets. ProbSparse Self-attention offers a balanced approach for medium-length sequences, being relatively computationally efficient.

5. Conclusion

In this paper, we propose an Informer-based long series forecasting model, which aims to enhance the ability of Informer to capture long- and short-term dependencies and improve forecasting accuracy. The model follows an encoder-decoder structure. Within each encoder, the probabilistic patch sampling attention mechanism improves the model's ability to model local temporal dependencies by performing local attention computations within different patches. The dilated causal pooling layer, on the other hand, utilizes causal convolution operations to capture long-term correlations at different time scales in a time series effectively, thus enhancing the model's expressive power in time series modeling. The decoder part of the structure first performs nonlinear transformations and feature extraction on the input sequence to enhance the model's abstract representation of the input data. Next, the attention computation is optimized by focusing precisely on the key information in the sequence through a self-attention mechanism. This structure improves the accuracy of data processing while enhancing the model's understanding and representation of the relationships within the data, leading to more accurate performance in decoding tasks. We conducted experiments on multivariate and univariate time series forecasting tasks, and the results show that PMformer exhibits optimal performance in both MAE and MSE metrics compared to six benchmark models, such as PatchTST and FEDformer. This suggests that our designed model can better capture the relationships within time series data and achieve more accurate forecasting.

CRediT authorship contribution statement

Yuewei Xue: Writing – original draft, Software, Conceptualization. **Shaopeng Guan:** Writing – review & editing, Supervision, Methodology. **Wanhai Jia:** Writing – review & editing, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] H. Abbasimehr, R. Paki, Improving time series forecasting using lstm and attention models, J. Ambient Intell. Humaniz. Comput. 13 (2022) 673–691, https://doi.org/10.1007/s12652-020-02761-x.
[2] A.O. Aseeri, Effective rnn-based forecasting methodology design for improving short-term power load forecasts: application to large-scale power-grid time series, J. Comput. Sci. 68 (2023) 101984, https://doi.org/10.1016/j.jocs.2023.101984.
[3] A. Babii, E. Ghysels, J. Striaukas, Machine learning time series regressions with an application to nowcasting, J. Bus. Econ. Stat. 40 (2022) 1094–1106, https://doi.org/10.1080/07350015.2021.1899933.
[4] L. Cascone, S. Sadiq, S. Ullah, S. Mirjalili, H.U.R. Siddiqui, M. Umer, Predicting household electric power consumption using multi-step time series with convolutional lstm, Big Data Res. 31 (2023) 100360, https://doi.org/10.1016/j.bdr.2022.100360.
[5] P. Chen, Y. Zhang, Y. Cheng, Y. Shu, Y. Wang, Q. Wen, B. Yang, C. Guo, Pathformer: multi-scale transformers with adaptive pathways for time series forecasting, arXiv preprint arXiv:2402.05956, https://doi.org/10.48550/arXiv.2402.05956, 2024.
[6] Z. Chen, M. Ma, T. Li, H. Wang, C. Li, Long sequence time-series forecasting with deep learning: a survey, Inf. Fusion 97 (2023) 101819, https://doi.org/10.1016/j.inffus.2023.101819.
[7] R. Corizzo, J. Rosen, Stock market prediction with time series data and news headlines: a stacking ensemble approach, J. Intell. Inf. Syst. 62 (2024) 27–56, https://doi.org/10.1007/s10844-023-00804-1.
[8] A.K. Dubey, A. Kumar, V. García-Díaz, A.K. Sharma, K. Kanhaiya, Study and analysis of sarima and lstm in forecasting time series data, Sustain. Energy Technol. Assess. 47 (2021) 101474, https://doi.org/10.1016/j.seta.2021.101474.
[9] Y. Feng, J.S. Kim, J.W. Yu, K.C. Ri, S.J. Yun, I.N. Han, Z. Qi, X. Wang, Spatiotemporal informer: a new approach based on spatiotemporal embedding and attention for air quality forecasting, Environ. Pollut. 336 (2023) 122402, https://doi.org/10.1016/j.envpol.2023.122402.
[10] P. Gao, W. Du, Q. Lei, J. Li, S. Zhang, N. Li, Ndvi forecasting model based on the combination of time series decomposition and cnn–lstm, Water Resour. Manag. 37 (2023) 1481–1497, https://doi.org/10.1007/s11269-022-03419-3.
[11] F. Jan, I. Shah, S. Ali, Short-term electricity prices forecasting using functional time series analysis, Energies 15 (2022) 3423, https://doi.org/10.3390/en15093423.
[12] Z.S. Khozani, F.B. Banadkooki, M. Ehteram, A.N. Ahmed, A. El-Shafie, Combining autoregressive integrated moving average with long short-term memory neural network and optimisation algorithms for predicting ground water level, J. Clean. Prod. 348 (2022) 131224, https://doi.org/10.1016/j.jclepro.2022.131224.
[13] N. Kitaev, Ł. Kaiser, A. Levskaya, Reformer: the efficient transformer, arXiv preprint arXiv:2001.04451, https://doi.org/10.48550/arXiv.2001.04451, 2020.
[14] F. Li, Z. Wan, T. Koch, G. Zan, M. Li, Z. Zheng, B. Liang, Improving the accuracy of multi-step prediction of building energy consumption based on eemd-pso-informer and long-time series, Comput. Electr. Eng. 110 (2023) 108845, https://doi.org/10.1016/j.compeleceng.2023.108845.
[15] S. Liu, H. Yu, C. Liao, J. Li, W. Lin, A.X. Liu, S. Dustdar, Pyraformer: low-complexity pyramidal attention for long-range time series modeling and forecasting, in: International Conference on Learning Representations, 2021, https://openreview.net/forum?id=0EXmFzUn5I.
[16] Z. Liu, Y. Cao, H. Xu, Y. Huang, Q. He, X. Chen, X. Tang, X. Liu, Hidformer: hierarchical dual-tower transformer using multi-scale mergence for long-term time series forecasting, Expert Syst. Appl. 239 (2024) 122412, https://doi.org/10.1016/j.eswa.2023.122412.
[17] R.P. Masini, M.C. Medeiros, E.F. Mendes, Machine learning advances for time series forecasting, J. Econ. Surv. 37 (2023) 76–111, https://doi.org/10.1111/joes.12429.
[18] M.A. Morid, O.R.L. Sheng, J. Dunbar, Time series prediction using deep learning methods in healthcare, ACM Trans. Manag. Inf. Syst. 14 (2023) 1–29, https://doi.org/10.1145/3531326.
[19] S. Mozaffari, S. Javadi, H.K. Moghaddam, T.O. Randhir, Forecasting groundwater levels using a hybrid of support vector regression and particle swarm optimization, Water Resour. Manag. 36 (2022) 1955–1972, https://doi.org/10.1007/s11269-022-03118-z.
[20] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, arXiv preprint arXiv:2211.14730, https://doi.org/10.48550/arXiv.2211.14730, 2022.
[21] Y. Ning, H. Kazemi, P. Tahmasebi, A comparative machine learning study for time series oil production forecasting: arima, lstm, and prophet, Comput. Geosci. 164 (2022) 105126, https://doi.org/10.1016/j.cageo.2022.105126.

[22] R.M. Pattanayak, H.S. Behera, S. Panigrahi, A novel high order hesitant fuzzy time series forecasting by using mean aggregated membership value with support vector machine, Inf. Sci. 626 (2023) 494–523, https://doi.org/10.1016/j.ins.2023.01.075.
[23] L.H. Torres, B. Ribeiro, J.P. Arrais, Multi-scale cross-attention transformer via graph embeddings for few-shot molecular property prediction, Appl. Soft Comput. 153 (2024) 111268, https://doi.org/10.1016/j.asoc.2024.111268.
[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017), https://doi.org/10.48550/arXiv.1706.03762.
[25] B. Wang, X. Liu, M. Chi, Y. Li, Bayesian network based probabilistic weighted high-order fuzzy time series forecasting, Expert Syst. Appl. 237 (2024) 121430, https://doi.org/10.1016/j.eswa.2023.121430.
[26] Y. Wang, H. Long, L. Zheng, J. Shang, Graphformer: adaptive graph correlation transformer for multivariate long sequence time series forecasting, Knowl.-Based Syst. 285 (2024) 111321, https://doi.org/10.1016/j.knosys.2023.111321.
[27] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, Adv. Neural Inf. Process. Syst. 34 (2021) 22419–22430, https://doi.org/10.48550/arXiv.2106.13008.
[28] X. Xiang, X. Li, Y. Zhang, J. Hu, A short-term forecasting method for photovoltaic power generation based on the tcn-ecanet-gru hybrid model, Sci. Rep. 14 (2024) 6744, https://doi.org/10.1038/s41598-024-56751-6.
[29] Z. Xiao, X. Huang, J. Liu, C. Li, Y. Tai, A novel method based on time series ensemble model for hourly photovoltaic power prediction, Energy 276 (2023) 127542, https://doi.org/10.1016/j.energy.2023.127542.
[30] X. Xu, Y. Zhang, A Gaussian process regression machine learning model for forecasting retail property prices with Bayesian optimizations and cross-validation, Decis. Anal. J. 8 (2023) 100267, https://doi.org/10.1016/j.dajour.2023.100267.
[31] C. Yang, Y. Wang, B. Yang, J. Chen, Graformer: a gated residual attention transformer for multivariate time series forecasting, Neurocomputing 581 (2024) 127466, https://doi.org/10.1016/j.neucom.2024.127466.
[32] X. Yang, L. He, Z. Zhang, Q. Luo, A chaotic discriminant algorithm for arrival traffic flow time series based on improved alternative data method, J. Internet Technol. 24 (2023) 1131–1139, https://doi.org/10.53106/160792642023092405011.
[33] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting?, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2023, pp. 11121–11128.
[34] P. Zeng, G. Hu, X. Zhou, S. Li, P. Liu, S. Liu, Muformer: a long sequence time-series forecasting model based on modified multi-head attention, Knowl.-Based Syst. 254 (2022) 109584, https://doi.org/10.1016/j.knosys.2022.109584.
[35] L. Zhao, H. Yuan, K. Xu, J. Bi, B.H. Li, Hybrid network attack prediction with Savitzky–Golay filter-assisted informer, Expert Syst. Appl. 235 (2024) 121126, https://doi.org/10.1016/j.eswa.2023.121126.
[36] L.T. Zhao, Y. Li, X.H. Chen, L.Y. Sun, Z.Y. Xue, Mftm-informer: a multi-step prediction model based on multivariate fuzzy trend matching and informer, Inf. Sci. 681 (2024) 121268, https://doi.org/10.1016/j.ins.2024.121268.
[37] Q. Zheng, X. Tian, Z. Yu, B. Jin, N. Jiang, Y. Ding, M. Yang, A. Elhanashi, S. Saponara, K. Kpalma, Application of complete ensemble empirical mode decomposition based multi-stream informer (ceemd-msi) in pm2. 5 concentration long-term prediction, Expert Syst. Appl. 245 (2024) 123008, https://doi.org/10.1016/j.eswa.2023.123008.
[38] S. Zhong, S. Song, G. Li, W. Zhuo, Y. Liu, S.H.G. Chan, A multi-scale decomposition mlp-mixer for time series analysis, arXiv preprint arXiv:2310.11959, https://doi.org/10.48550/arXiv.2310.11959, 2023.
[39] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2021, pp. 11106–11115.
[40] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: frequency enhanced decomposed transformer for long-term series forecasting, in: International Conference on Machine Learning, in: PMLR, 2022, pp. 27268–27286.